Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: «Современные платформы программирования»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

Веб-приложение для изучения иностранных языков

БГУИР КП 6-05-0612-01 123 ПЗ

Студент Ушаков А.Д.

Руководитель Петровская В.В.

Минск 2026

СОДЕРЖАНИЕ

Введение 7
1 Анализ предметной области и существующих аналогов 8
1.1 Обзор аналогов 8
1.2 Сравнительный анализ аналогов 12
1.3 Постановка задачи 13
2 Анализ требований к программному средству и разработка функциональных требований 14
2.1 Назначение программного средства 14
2.2 Функциональные требования 14
2.3 Нефункциональные требования 17
2.4 Требования к входным и выходным данным 18
2.5 Требования к интерфейсу 18
3 Проектирование программного средства 20
3.1 Архитектура программного средства 20
3.2 Проектирование базы данных 21
3.3 Проектирование программного интерфейса REST 23
3.4 Проектирование пользовательского интерфейса 24
3.5 Алгоритмы функционирования 24
4 Создание (конструирование) программного средства 33
4.1 Средства разработки 33
4.2 Реализация аутентификации и профиля 33
4.3 Проверка данных и справочники 34
4.4 Реализация словаря и уроков 35
4.5 Интеграция со сторонним сервисом перевода 35
4.6 Реализация уведомлений SSE 36
4.7 Модальные формы, выпадающие списки и озвучивание 37
4.8 Реализация тренировки слов 37
4.9 Адаптивная вёрстка 38
5 Тестирование, проверка работоспособности и анализ полученных результатов 39

5.1 Виды тестирования 39
5.2 Тест-кейсы 39
5.3 Анализ результатов 42
6 Руководство по установке и использованию 43
6.1 Установка и запуск 43
6.2 Руководство пользователя 44
Заключение 54
Список использованных источников 55
ПРИЛОЖЕНИЕ А (обязательное) Листинги программного средства 56

ВВЕДЕНИЕ

Изучение иностранных языков является важным условием личного и профессионального развития, а цифровые технологии делают этот процесс более доступным и удобным. Современные веб-приложения позволяют пользователю заниматься в удобное время, работать с собственными учебными материалами и отслеживать прогресс. В связи с этим разработка веб-приложения для изучения иностранных языков является актуальной задачей.

Целью курсового проекта является создание веб-приложения для изучения иностранных языков, которое обеспечивает регистрацию и авторизацию пользователей, работу с учебными материалами, поиск, фильтрацию и сортировку записей, тренировку слов и отслеживание прогресса, двусторонний перевод через сторонний сервис, озвучивание слов и отправку уведомлений. Для достижения цели необходимо проанализировать предметную область и существующие аналоги, сформулировать требования к программному средству, спроектировать архитектуру и базу данных, реализовать клиентскую и серверную части, провести тестирование и подготовить руководство по установке и использованию.

При разработке используются Next.js с TypeScript, программный интерфейс REST на обработчиках маршрутов Route Handlers, MongoDB с Mongoose, шифрование паролей с помощью bcrypt, механизм событий, отправляемых сервером (Server-Sent Events, SSE), сторонний сервис перевода MyMemory, синтез речи браузера, а также Git и GitHub. Пользовательский интерфейс приложения должен быть русскоязычным, адаптивным и удобным как на персональных компьютерах, так и на мобильных устройствах [1, 2].

Практическая значимость проекта заключается в создании инструмента, который позволяет систематизировать изучение иностранного языка, хранить слова и уроки, тренировать лексику и отслеживать прогресс. В первом разделе выполнен анализ предметной области и аналогов. Во втором разделе сформулированы требования к программному средству. В третьем разделе приведено проектирование архитектуры, данных, пользовательского интерфейса и алгоритмов. В четвёртом разделе описано конструирование программного средства. В пятом разделе представлены результаты тестирования. В шестом разделе приведено руководство по установке и использованию.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ АНАЛОГОВ

1.1 Обзор аналогов

Изучение иностранного языка включает запоминание слов, освоение грамматики, развитие навыков чтения, аудирования, письма и речи. Веб-приложения позволяют организовать этот процесс гибко и индивидуально. Для определения особенностей собственной разработки необходимо рассмотреть существующие аналоги, выявить их сильные и слабые стороны, а затем сформулировать требования к проектируемому программному средству.

1.1.1 Программное средство Duolingo. Среди наиболее известных решений для изучения иностранных языков можно выделить Duolingo, Lingualeo, Quizlet и Anki. Они различаются по функциональности, способам подачи материала, наличию геймификации и возможностям работы с пользовательским контентом.

Duolingo представляет собой популярную платформу с короткими уроками, упражнениями на перевод, аудирование и выбор правильного ответа. Система использует баллы, уровни, достижения и серии ежедневных занятий, что повышает мотивацию. К преимуществам относятся бесплатный базовый доступ, понятный интерфейс и большое количество языков. Однако платформа ориентирована на стандартизированные курсы, слабо поддерживает создание собственных учебных материалов и ограниченно представляет сложную грамматику и профессиональную лексику. Интерфейс веб-приложения Duolingo показан на рисунке 1.1.

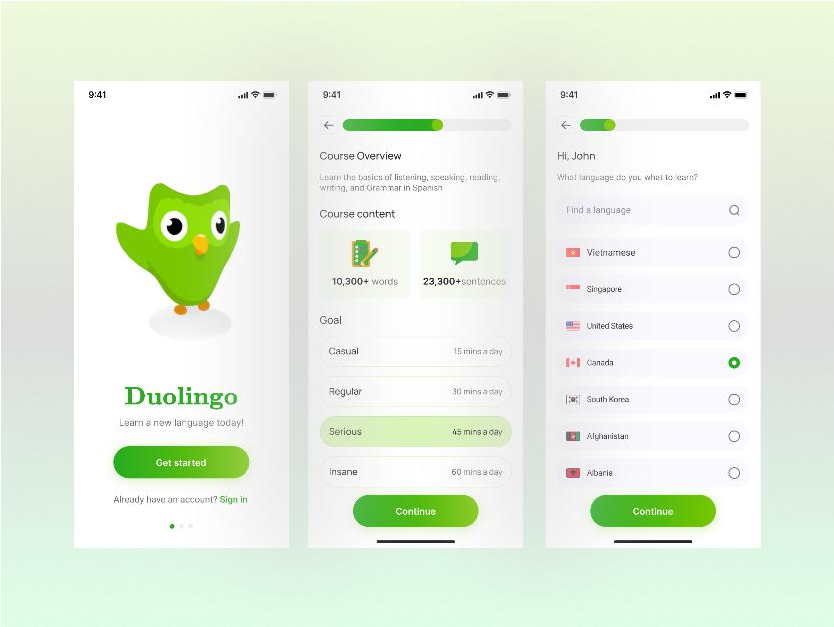


Рисунок 1.1 – Интерфейс веб-приложения Duolingo

1.1.2 Программное средство Lingualeo. Lingualeo ориентирован в том числе на русскоязычных пользователей и предлагает интерактивные упражнения, видео, тексты, грамматические курсы и персональный словарь. Пользователь может добавлять незнакомые слова и тренировать их в разных режимах. К достоинствам относятся русскоязычный интерфейс, разнообразие тренировок и работа с реальными материалами. К недостаткам можно отнести ограничения бесплатной версии, преимущественную ориентацию на английский язык и ограниченные возможности настройки индивидуального учебного плана. Интерфейс приложения Lingualeo показан на рисунке 1.2.

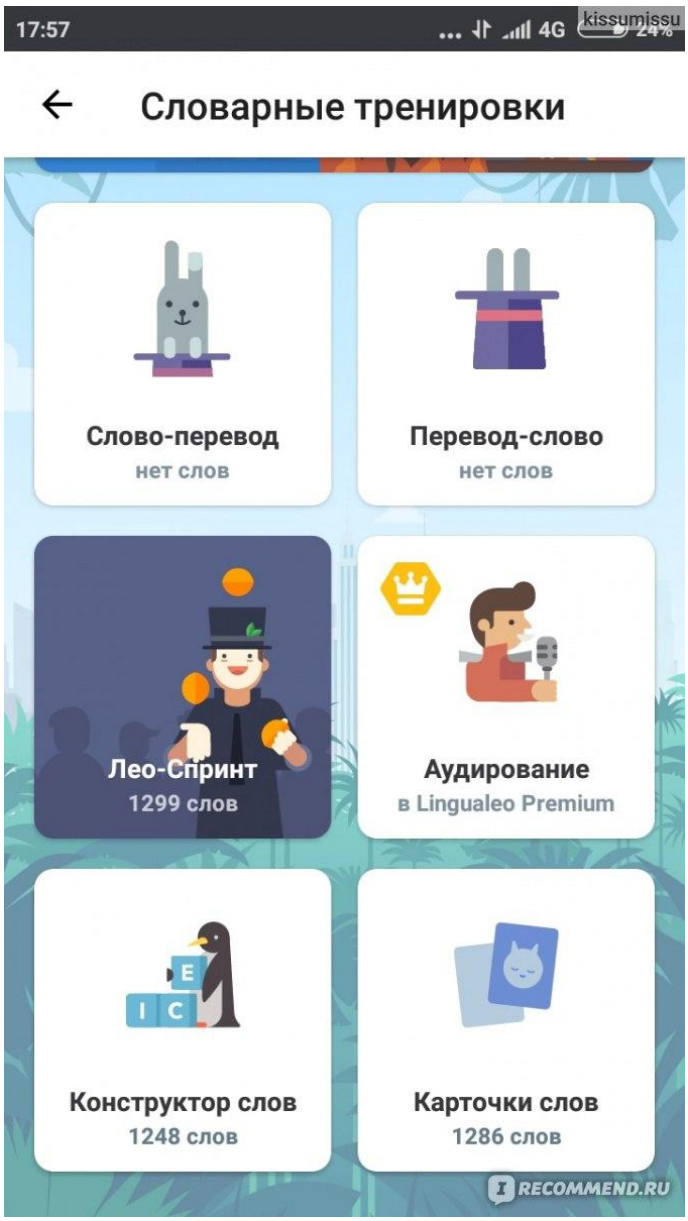


Рисунок 1.2 – Интерфейс веб-приложения Lingualeo

1.1.3 Программное средство Quizlet. Quizlet предназначен для создания и использования учебных карточек, тестов и игровых заданий. Пользователи могут формировать собственные наборы карточек, добавлять термины и определения, использовать режимы заучивания и проверки знаний. Сервис гибок и удобен для повторения, но не является полноценным курсом иностранного языка, поскольку в нём мало внимания уделяется грамматике, аудированию и разговорной практике. Интерфейс Quizlet показан на рисунке 1.3.

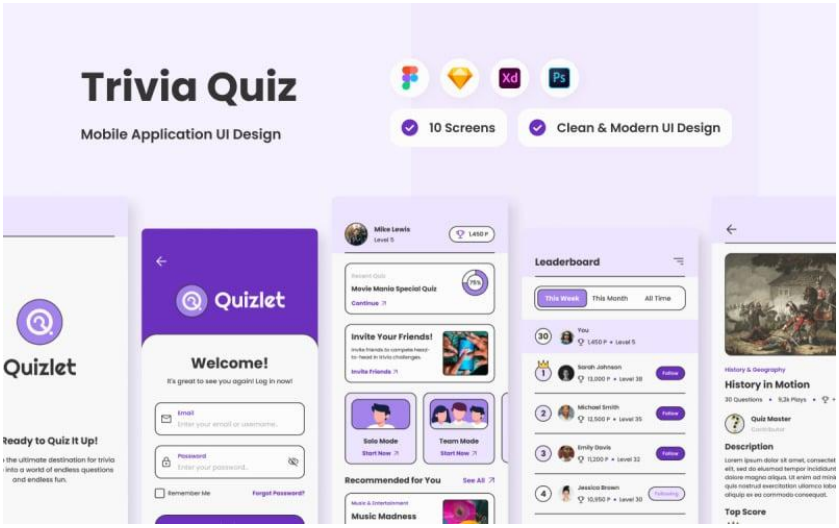


Рисунок 1.3 – Интерфейс веб-приложения Quizlet

1.1.4 Программное средство Anki. Anki использует карточки и интервальное повторение для эффективного запоминания информации. Приложение поддерживает создание собственных колод, импорт и экспорт данных, теги, статистику и синхронизацию. К преимуществам относятся высокая эффективность повторения и гибкость настройки. Однако интерфейс может быть сложным для начинающих, отсутствует готовый структурированный курс языка, а создание качественных материалов требует времени и методических знаний. Интерфейс Anki показан на рисунке 1.4.

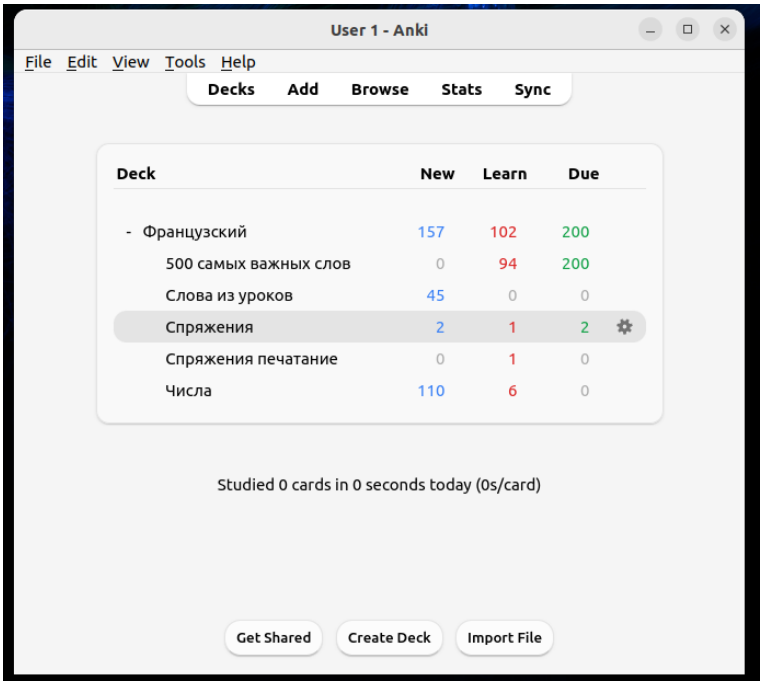


Рисунок 1.4 – Интерфейс приложения Anki

1.2 Сравнительный анализ аналогов

Рассмотренные приложения обладают развитым функционалом, но каждое имеет ограничения. Duolingo и Lingualeo предлагают структурированные уроки и элементы геймификации, однако значительная часть возможностей доступна по подписке, а работа с собственными материалами ограничена. Quizlet и Anki хорошо подходят для запоминания и повторения, но не дают полноценного курса иностранного языка и требуют самостоятельного создания контента. Кроме того, не все аналоги обеспечивают удобную интеграцию перевода, уведомления и адаптивный личный кабинет с разделами для слов, уроков и прогресса. Сравнение аналогов по ключевым признакам приведено в таблице 1.1.

Таблица 1.1 – Сравнение аналогов проектируемого программного средства

Признак	Duolingo	Lingualeo	Quizlet	Anki
Собственные материалы	ограничено	частично	да	да
Русскоязычный интерфейс	частично	да	частично	частично
Поиск и сортировка записей	ограничено	частично	да	да
Интеграция перевода	ограничено	частично	нет	нет

Признак	Duolingo	Lingualeo	Quizlet	Anki
Уведомления в реальном времени	нет	нет	нет	нет
Открытый исходный код проекта	нет	нет	нет	да

Анализ таблицы 1.1 показывает, что ни один из рассмотренных аналогов одновременно не сочетает полный набор операций создания, просмотра, изменения и удаления учебных материалов, встроенный двусторонний перевод, уведомления и развёртывание в виде собственного веб-приложения. Это подтверждает целесообразность разработки программного средства Lingua, которое объединит необходимые функции в единой системе.

1.3 Постановка задачи

На основе анализа предметной области и существующих аналогов необходимо разработать веб-приложение для изучения иностранных языков с использованием Next.js и TypeScript, программного интерфейса REST на обработчиках маршрутов Route Handlers, MongoDB с Mongoose, bcrypt для шифрования паролей и событий, отправляемых сервером, для уведомлений [2–4]. Приложение должно обеспечивать регистрацию и вход пользователей, проверку входящих данных, создание, редактирование, удаление и просмотр учебных материалов, работу со словами и уроками, поиск, фильтрацию и сортировку записей, личный кабинет с разделами для изучения слов, уроков и прогресса, тренировку слов, двусторонний перевод через сторонний сервис, озвучивание слов, управление профилем, отправку уведомлений через SSE, адаптивную вёрстку и хранение данных в MongoDB. Проект должен вестись в Git с размещением репозитория на GitHub. В результате предполагается создать полноценное веб-приложение с русскоязычным интерфейсом, которое позволит пользователю систематизировать изучение иностранного языка, работать с собственными материалами и отслеживать свои результаты.

2 АНАЛИЗ ТРЕБОВАНИЙ К ПРОГРАММНОМУ СРЕДСТВУ И РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

2.1 Назначение программного средства

Программное средство Lingua предназначено для индивидуальной работы пользователя с учебными материалами по иностранному языку. Система должна позволять регистрировать учётную запись, входить в личный кабинет, вести словарь, формировать уроки из собственных текстов и слов, тренировать слова, контролировать прогресс, переводить слова и фразы в обе стороны, прослушивать произношение и получать уведомления о значимых действиях. Программное средство ориентировано на эксплуатацию в браузере и разворачивается в контейнерной среде Docker.

Потребителем является обучающийся, который самостоятельно формирует словарный запас и набор уроков. Административная роль в рамках курсового проекта не выделяется: каждый пользователь работает только со своими данными.

2.2 Функциональные требования

На основании задания на курсовое проектирование и результатов анализа аналогов сформулированы функциональные требования (ФТ). Программное средство должно обеспечивать:

– ФТ-1 регистрацию пользователя по имени, почте, паролю, родному и изучаемому языкам;

– ФТ-2 вход в систему и завершение сессии;

– ФТ-3 хранение сессии на основе JWT в защищённом файле cookie;

– ФТ-4 создание, просмотр, изменение и удаление словарных записей;

– ФТ-5 создание, просмотр, изменение и удаление уроков с привязкой слов из словаря;

– ФТ-6 поиск записей по тексту, фильтрацию по тегу, языку и уровню, сортировку по выбранному полю;

– ФТ-7 выбор языка, части речи, тегов и уровня из выпадающих списков;

– ФТ-8 проверку входящих данных на стороне сервера;

– ФТ-9 шифрование паролей алгоритмом bcrypt;

– ФТ-10 отображение прогресса по словарю и урокам;

– ФТ-11 тренировку слов в направлениях «слово – перевод» и «перевод – слово» по всему словарю или по словам выбранного урока с изменением уровня усвоения;

– ФТ-12 двусторонний перевод через сторонний сервис: с изучаемого языка на родной и обратно;

– ФТ-13 озвучивание слов и примеров средствами синтеза речи браузера;

– ФТ-14 изменение профиля и смену пароля;

– ФТ-15 доставку уведомлений по протоколу SSE и отметку уведомлений прочитанными;

– ФТ-16 адаптивное отображение интерфейса на узких экранах;

– ФТ-17 разграничение данных по идентификатору пользователя.

1) Комментарий к ФТ-1 и ФТ-2. Регистрация требует имени длиной не менее двух символов, корректной почты и пароля длиной от 8 до 72 символов. Родной язык выбирается из списка «русский, белорусский, украинский», изучаемый – из списка «английский, немецкий, французский, испанский». Повторная регистрация с той же почтой должна отклоняться. Вход выполняется по почте и паролю; при несовпадении возвращается ошибка без уточнения, какое именно поле неверно.

2) Комментарий к ФТ-4 и ФТ-7. Словарная запись содержит слово, перевод, язык, а также необязательные транскрипцию, часть речи, пример употребления, заметку, теги и уровень усвоения от 0 до 5. Часть речи выбирается из десяти значений (существительное, глагол, прилагательное и т. д.), тег – из списка уже использованных и типовых тегов либо вводится вручную. Теги приводятся к нижнему регистру, повторы удаляются, на одно слово допускается не более 12 тегов. Форма открывается в модальном окне поверх списка, поэтому список слов виден сразу после перехода в раздел.

3) Комментарий к ФТ-5. Урок содержит название от 2 до 160 символов, текст от 10 до 8000 символов, язык, уровень A1–C2 и список слов. В урок можно добавить только слова текущего пользователя; при удалении слова оно автоматически исключается из всех уроков. Урок открывается в окне чтения, откуда можно перейти к тренировке его слов.

4) Комментарий к ФТ-11. При тренировке пользователь вводит ответ с клавиатуры. Ответ сравнивается без учёта регистра и лишних пробелов с каждым из вариантов перевода, перечисленных через запятую или точку с запятой. Верный ответ повышает уровень усвоения на единицу, неверный – понижает, значение остаётся в диапазоне от 0 до 5. Доступны подсказка и пропуск слова.

5) Комментарий к ФТ-12. Перевод выполняется через HTTP-запрос к сервису MyMemory. Пара языков проверяется на сервере: оба языка должны

входить в список поддерживаемых и не совпадать. В форме слова доступны кнопки перевода в обе стороны, в разделе «Переводчик» направление меняется кнопкой обмена языков. При недоступности внешнего сервиса клиент получает сообщение об ошибке, локальные данные не повреждаются [5].

6) Комментарий к ФТ-13 и ФТ-14. Озвучивание использует голос, соответствующий языку записи. Если браузер не поддерживает синтез речи, кнопка становится неактивной с поясняющей подсказкой. В профиле изменяются имя, родной и изучаемый языки; смена пароля требует ввода текущего пароля, новый пароль должен отличаться от текущего.

7) Комментарий к ФТ-15. Клиент кабинета открывает поток text/event-stream. При создании, изменении и удалении записей сервер сохраняет уведомление в MongoDB и сразу отправляет событие подключённому клиенту. Всплывающее сообщение скрывается через 8 с или по кнопке закрытия, команда «Прочитать все» отмечает журнал прочитанным и закрывает все всплывающие сообщения.

Взаимодействие пользователя и внешнего сервиса перевода с системой показано на диаграмме вариантов использования (рисунок 2.1). Основные варианты связаны с пользователем напрямую, вспомогательные отражены отношениями «include» (обязательное включение) и «extend» (необязательное расширение).

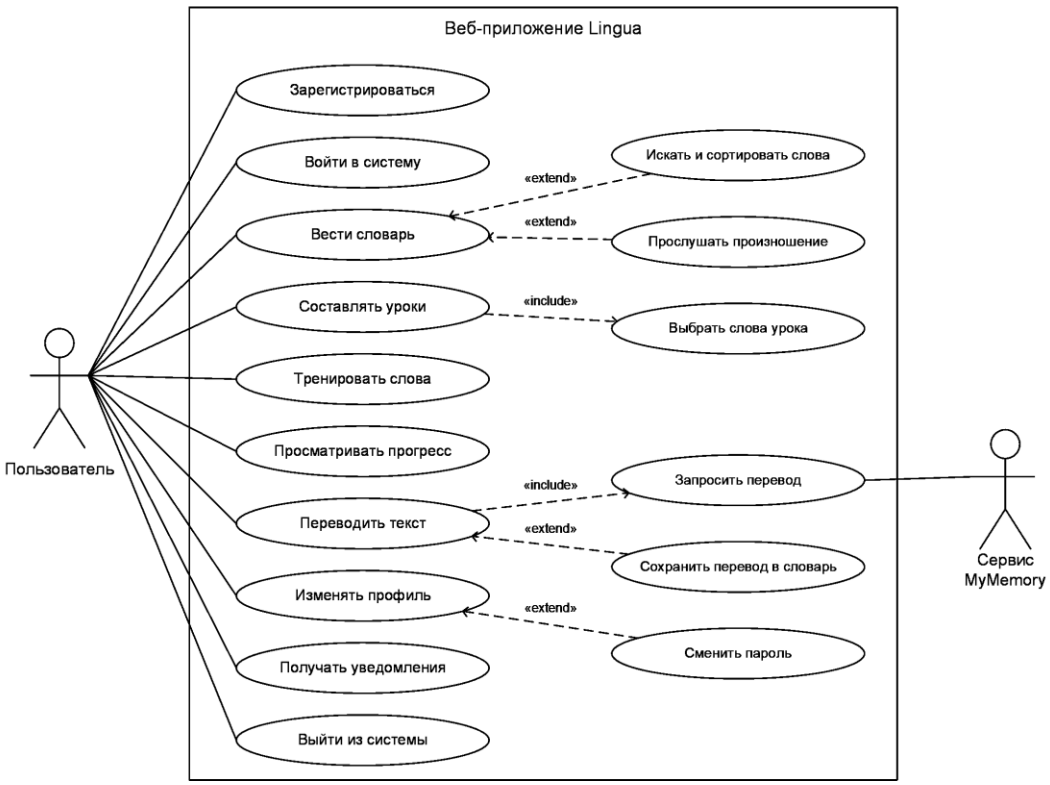


Рисунок 2.1 – Диаграмма вариантов использования

2.3 Нефункциональные требования

К программному средству предъявляются следующие нефункциональные требования:

– НФТ-1 клиентская и серверная части реализуются на TypeScript в рамках Next.js;

– НФТ-2 программный интерфейс строится по принципам REST на обработчиках маршрутов Route Handlers;

– НФТ-3 данные хранятся в MongoDB и описываются схемами Mongoose [3];

– НФТ-4 пользовательский интерфейс выполняется на русском языке;

– НФТ-5 запуск выполняется командой docker compose up;

– НФТ-6 исходный код сопровождается репозиторием Git;

– НФТ-7 время ответа локального сервера при типовых операциях с записями не должно превышать нескольких секунд на учебной нагрузке;

– НФТ-8 обращение к внешнему сервису перевода ограничивается тайм-аутом 10 с.

2.4 Требования к входным и выходным данным

Входными данными являются заполняемые пользователем формы регистрации, входа, профиля, словаря, уроков, тренировки и перевода, а также параметры поиска, фильтрации и сортировки в строке запроса. Выходными данными являются ответы сервера в формате JSON и HTML-страницы приложения. Форматы основных запросов приведены в таблице 2.1.

Таблица 2.1 – Основные входные и выходные данные

Операция	Вход	Выход
Регистрация	JSON: name, email, password, nativeLanguage, targetLanguage	JSON: user; cookie сессии
Вход	JSON: email, password	JSON: user; cookie сессии
Список слов	q, tag, language, lesson, sort, order, limit	JSON: items[]
Создание слова	JSON полей записи	JSON: item; уведомление SSE
Создание урока	JSON: title, content, language, level, words[]	JSON: item со списком слов
Перевод	JSON: text, from, to	JSON: translation, from, to
Изменение профиля	JSON: name, nativeLanguage, targetLanguage	JSON: user
Смена пароля	JSON: currentPassword, newPassword	JSON: ok
Поток уведомлений	cookie сессии	text/event-stream

Некорректные данные должны отклоняться с кодом 400, отсутствие сессии – с кодом 401, обращение к отсутствующей или чужой записи – с кодом 404, конфликт почты – с кодом 409, ошибка внешнего сервиса – с кодом 502.

2.5 Требования к интерфейсу

Пользовательский интерфейс должен содержать стартовую страницу, формы входа и регистрации, личный кабинет с разделами «Обзор», «Слова», «Уроки», «Прогресс», «Переводчик» и «Профиль». В шапке кабинета отображаются название приложения, имя пользователя и команда выхода. Список слов и карточки уроков должны быть видны сразу после открытия раздела, а формы создания и редактирования – открываться в модальном окне с переводом фокуса на первое поле. Удаление записей выполняется после подтверждения в отдельном окне. Для языка, части речи, тегов и

уровня используются выпадающие списки. Раздел «Прогресс» отображает количественные показатели, гистограмму усвоения и форму тренировки. Уведомления показываются в журнале кабинета и во всплывающих сообщениях. Цветовое оформление выполняется в спокойной учебной гамме без сторонних библиотек компонентов пользовательского интерфейса.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

3.1 Архитектура программного средства

Программное средство строится как одно приложение Next.js, в котором клиентские страницы и серверные обработчики маршрутов Route Handlers размещаются в общем проекте. Такой подход соответствует заданию: разработку ведут на TypeScript для клиентской и серверной частей, а программный интерфейс реализуют по принципам REST [2]. Внешними узлами являются СУБД MongoDB и сервис перевода MyMemory. Состав программных частей:

– слой представления – страницы App Router, модальные формы, таблицы и карточки кабинета;

– слой прикладного программного интерфейса – обработчики /api/auth, /api/words, /api/lessons, /api/progress, /api/translate, /api/profile, /api/notifications;

– слой данных – модели Mongoose User, Word, Lesson, Notice;

– слой интеграции – HTTP-клиент перевода и диспетчер SSE.

Клиент обращается к серверу того же происхождения, поэтому файл cookie сессии передаётся браузером автоматически. Размещение компонентов по узлам показано на диаграмме развёртывания (рисунок 3.1). Браузер пользователя обращается к контейнеру приложения по протоколу HTTP на порт 3000 и держит открытым поток SSE. Контейнер приложения связан с контейнером MongoDB внутренней сетью Docker Compose, данные СУБД сохраняются в именованном томе lingua_mongo. Обращение к сервису MyMemory выполняется только с сервера по протоколу HTTPS.

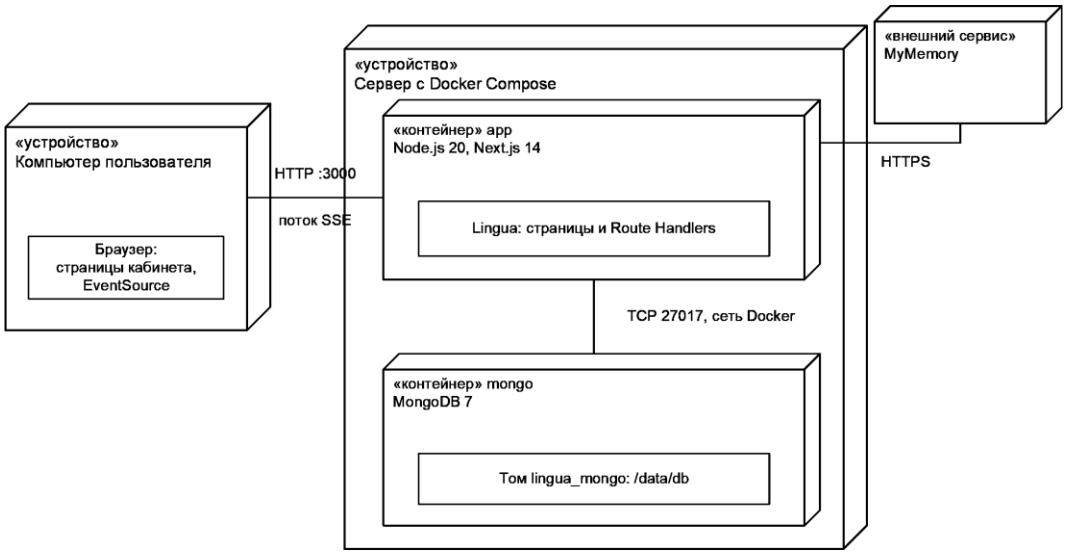


Рисунок 3.1 – Диаграмма развёртывания

В состав серверной части входят следующие модули:

– src/lib/db.ts – подключение к MongoDB с повторными попытками и заполнение демонстрационными данными;

– src/lib/auth.ts – формирование и проверка JWT, работа с файлом cookie;

– src/lib/validate.ts – проверка строк, почты, диапазона усвоения, тегов, идентификаторов и параметров списка;

– src/lib/catalog.ts – справочники языков, частей речи, уровней и типовых тегов;

– src/lib/words.ts и src/lib/lessons.ts – чтение полей и преобразование документов слов и уроков;

– src/lib/sse.ts и src/lib/notify.ts – подписка клиентов и публикация событий;

– модели предметной области в src/lib/models.

Справочники из catalog.ts используются одновременно сервером для проверки значений и клиентом для заполнения выпадающих списков, поэтому набор допустимых значений определяется в одном месте.

В состав клиентской части входят страницы:

– / – описание назначения Lingua и переход к регистрации;

– /login, /register – вход и создание учётной записи;

– /cabinet – обзор со сводкой, недавно добавленными словами и словами для повторения;

– /cabinet/words – словарь;

– /cabinet/lessons – уроки;

– /cabinet/progress – прогресс и тренировка;

– /cabinet/translator – переводчик;

– /cabinet/profile – профиль и смена пароля.

Общие элементы интерфейса вынесены в компоненты Modal (модальное окно), ConfirmDialog (подтверждение удаления), WordForm (форма слова), TagPicker (выбор тегов) и SpeakButton (кнопка озвучивания).

3.2 Проектирование базы данных

В качестве хранилища выбрана документная СУБД MongoDB. Для учебной системы со словарями и уроками естественна схема без жёстких соединений таблиц: каждая сущность хранится коллекцией документов, а связь с пользователем задаётся полем userId [3]. Состав коллекций приведён в таблице 3.1.

Таблица 3.1 – Коллекции базы данных Lingua

Коллекция	Назначение	Основные поля
users	учётные записи	name, email, passwordHash, nativeLanguage, targetLanguage
words	словарные записи	userId, term, translation, transcription, language, partOfSpeech, example, notes, tags, mastery
lessons	уроки	userId, title, content, language, level, words
notices	уведомления	userId, title, message, kind, read

Схема базы данных с сущностями, их полями и связями приведена на рисунке 3.2. Пользователь связан с записями words, lessons и notices отношением «один ко многим» через поле userId. Уроки и слова связаны отношением «многие ко многим»: урок хранит массив идентификаторов слов words, одно слово может входить в несколько уроков. При удалении слова его идентификатор исключается из всех уроков пользователя.

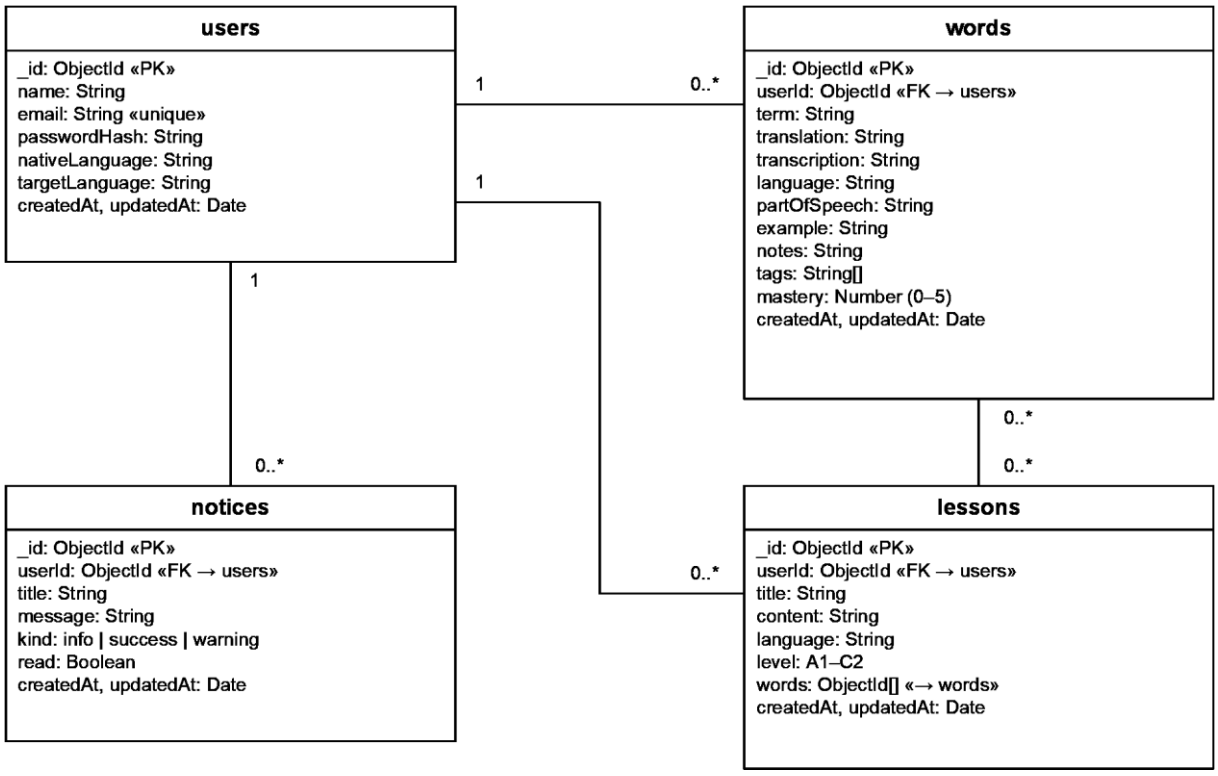


Рисунок 3.2 – Схема базы данных

Поле passwordHash хранит результат bcrypt, исходный пароль в базе не сохраняется [4]. Поле email снабжено уникальным индексом. Для выборки

записей пользователя создаётся индекс по userId, для слов дополнительно – составной индекс по userId и term. Поиск выполняется регулярным выражением без учёта регистра по полям слова, перевода, тегов, названия и текста урока; специальные символы в строке поиска экранируются.

3.3 Проектирование программного интерфейса REST

Взаимодействие клиента и сервера выполняется по REST. Идентификаторы передаются в пути, параметры поиска, фильтрации и сортировки – в строке запроса, тела запросов – в JSON. Перечень маршрутов приведён в таблице 3.2.

Таблица 3.2 – Маршруты программного интерфейса

Метод и путь	Назначение
POST /api/auth/register	регистрация
POST /api/auth/login	вход
POST /api/auth/logout	выход
GET /api/auth/me	текущий пользователь
GET, POST /api/words	список и создание слов
GET, PATCH, DELETE /api/words/[id]	чтение, изменение, удаление слова
GET /api/words/tags	список тегов пользователя
GET, POST /api/lessons	список и создание уроков
GET, PATCH, DELETE /api/lessons/[id]	чтение, изменение, удаление урока
GET /api/progress	сводка прогресса
POST /api/translate	перевод
PATCH /api/profile	изменение профиля
POST /api/profile/password	смена пароля
GET, PATCH /api/notifications	журнал уведомлений и отметка о прочтении
GET /api/notifications/stream	поток SSE
GET /api/health	проверка работоспособности

Список слов принимает параметры q (строка поиска), tag, language, lesson (слова выбранного урока), sort, order и limit. Список уроков принимает q, level, language, sort и order. Поле сортировки проверяется по белому списку, при недопустимом значении применяется сортировка по дате создания; limit ограничивается сотней записей.

Все маршруты, кроме регистрации, входа и проверки работоспособности, требуют действительной сессии. Ответ 401 возвращается, если файл cookie отсутствует или JWT неверен. Обращение к записи другого пользователя обрабатывается так же, как обращение к несуществующей записи, и завершается ответом 404.

3.4 Проектирование пользовательского интерфейса

Главное окно кабинета состоит из шапки, навигации по разделам, рабочей области и журнала уведомлений. Шапка содержит знак приложения, имя пользователя и кнопку выхода. Навигация выделяет активный раздел. В разделах «Слова» и «Уроки» над списком расположены кнопка создания и панель поиска с выпадающими списками фильтров и сортировки, ниже – таблица слов или карточки уроков. Список виден сразу после открытия раздела, а формы создания и редактирования открываются в модальном окне поверх списка. При открытии окна фокус переводится на первое поле формы, клавиша Escape и щелчок по затемнённому фону закрывают окно, после закрытия фокус возвращается к кнопке, которая открыла окно. Сохранённая запись кратковременно подсвечивается в списке.

Значения с ограниченным набором вариантов (язык, часть речи, тег, уровень, направление тренировки, источник слов) выбираются из выпадающих списков или переключателей, поэтому пользователь не может ввести недопустимое значение. Рядом со словами в таблице, в окне урока, на странице тренировки и в переводчике расположены кнопки озвучивания.

Цветовая схема строится на тёмно-синем основном цвете, тёплом бумажном фоне и акцентном терракотовом цвете для индикатора усвоения. Скругления блоков и тени используются умеренно, без декоративных иллюстраций, чтобы интерфейс оставался пригодным для учебной эксплуатации. На узкой ширине экрана сетки сворачиваются в один столбец, таблица получает горизонтальную прокрутку, а модальное окно прижимается к нижнему краю экрана и занимает всю его ширину.

3.5 Алгоритмы функционирования

Алгоритмы представлены блок-схемами по ГОСТ 19.701–90 [6]. На схемах применяются терминаторы «Начало» и «Конец», процессы, решения, а также блоки ввода-вывода в виде параллелограмма. Каждая схема описывает только одну сторону: серверную или клиентскую. На схемах серверной части формирование ответа с ошибкой является процессом, так как сервер не выводит сообщение пользователю, а возвращает код ответа. На схемах клиентской части вывод ошибки пользователю изображается блоком ввода-вывода. Схемы выполнены в Visio. Рамка и основная надпись используются только на чертеже формата А1. Работа пользователя с клиентской частью приложения в целом – от входа в кабинет до выхода – показана на этом чертеже «Схема программы»: на нём отражены переходы

между разделами кабинета и основные действия пользователя в каждом из них.

Запуск серверной части включает загрузку переменных окружения, подключение к MongoDB, инициализацию обработчиков маршрутов и прослушивание порта 3000. Последовательность показана на рисунке 3.3.

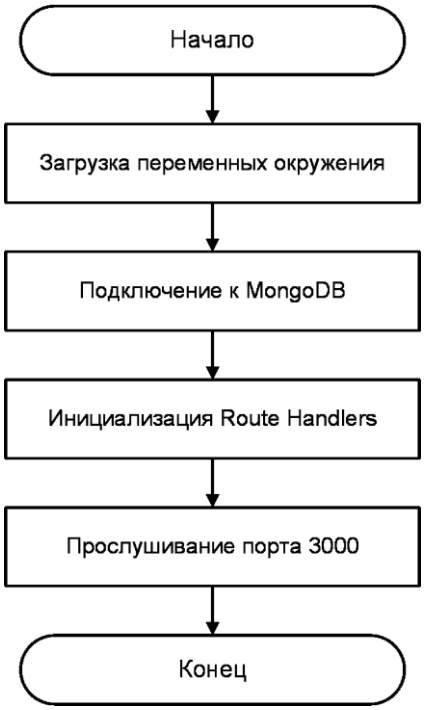


Рисунок 3.3 – Блок-схема запуска серверной части

Запуск клиентской части включает загрузку общего макета страниц, маршрутов кабинета, запрос текущего пользователя и подписку на поток уведомлений. Последовательность показана на рисунке 3.4.



Рисунок 3.4 – Блок-схема запуска клиентской части

Регистрация на сервере начинается с чтения полей запроса, затем выполняются проверка данных, проверка уникальности почты, хэширование пароля, создание документа, запись уведомления и выдача JWT. При некорректных данных сервер формирует ответ 400 со списком ошибок, при занятой почте – ответ 409. Схема приведена на рисунке 3.5.

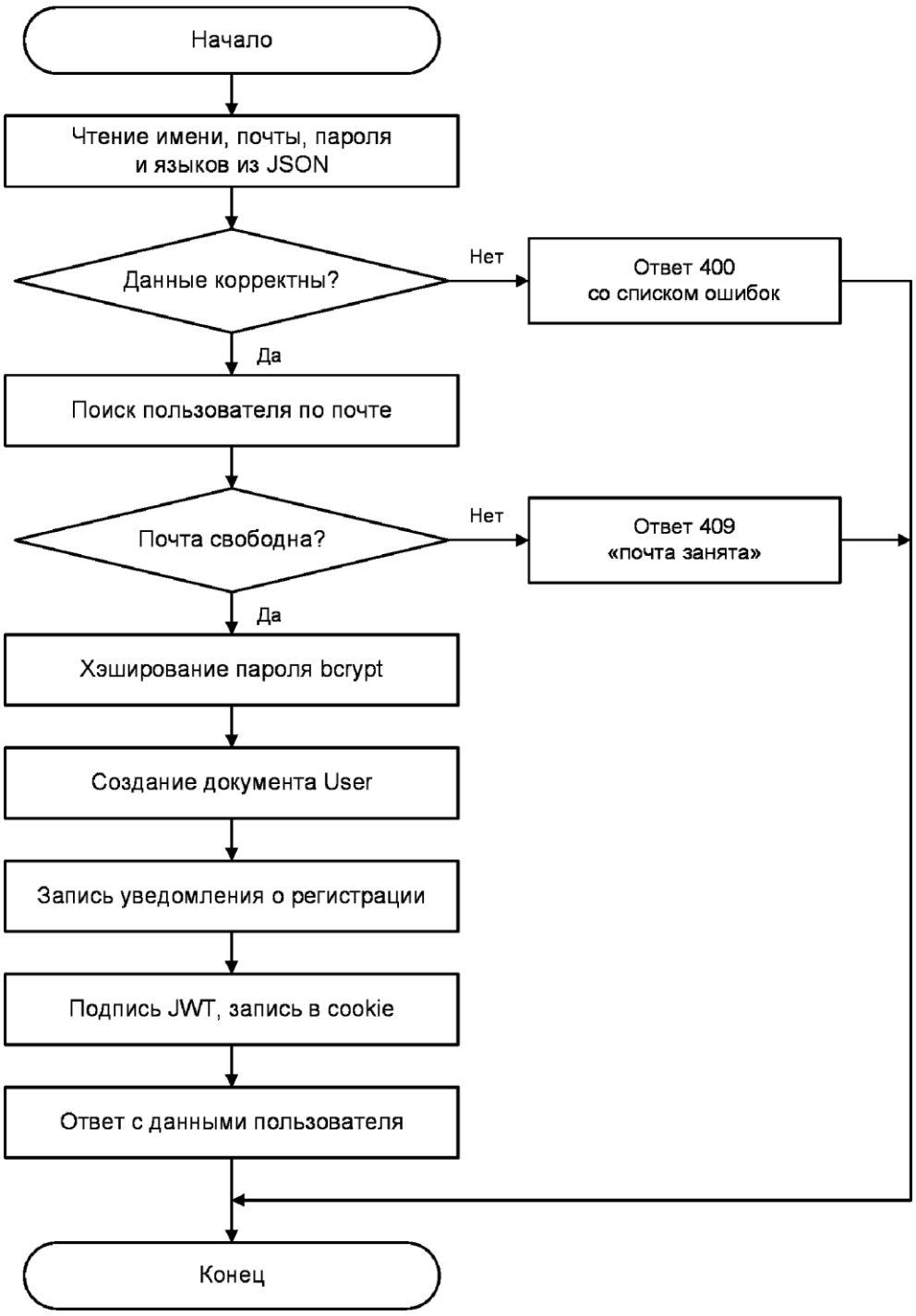


Рисунок 3.5 – Блок-схема регистрации пользователя на сервере

Создание и изменение словарной записи на клиенте начинается с открытия модального окна и перевода фокуса на поле «Слово». Пользователь заполняет форму и при необходимости запрашивает автоперевод. Если сервер отклонил запись, ошибки выводятся под полями и пользователь возвращается к вводу. После сохранения окно закрывается, таблица обновляется и выводится уведомление. Схема приведена на рисунке 3.6,

схема тренировки слов, которая также выполняется на клиенте, – на рисунке 3.7.

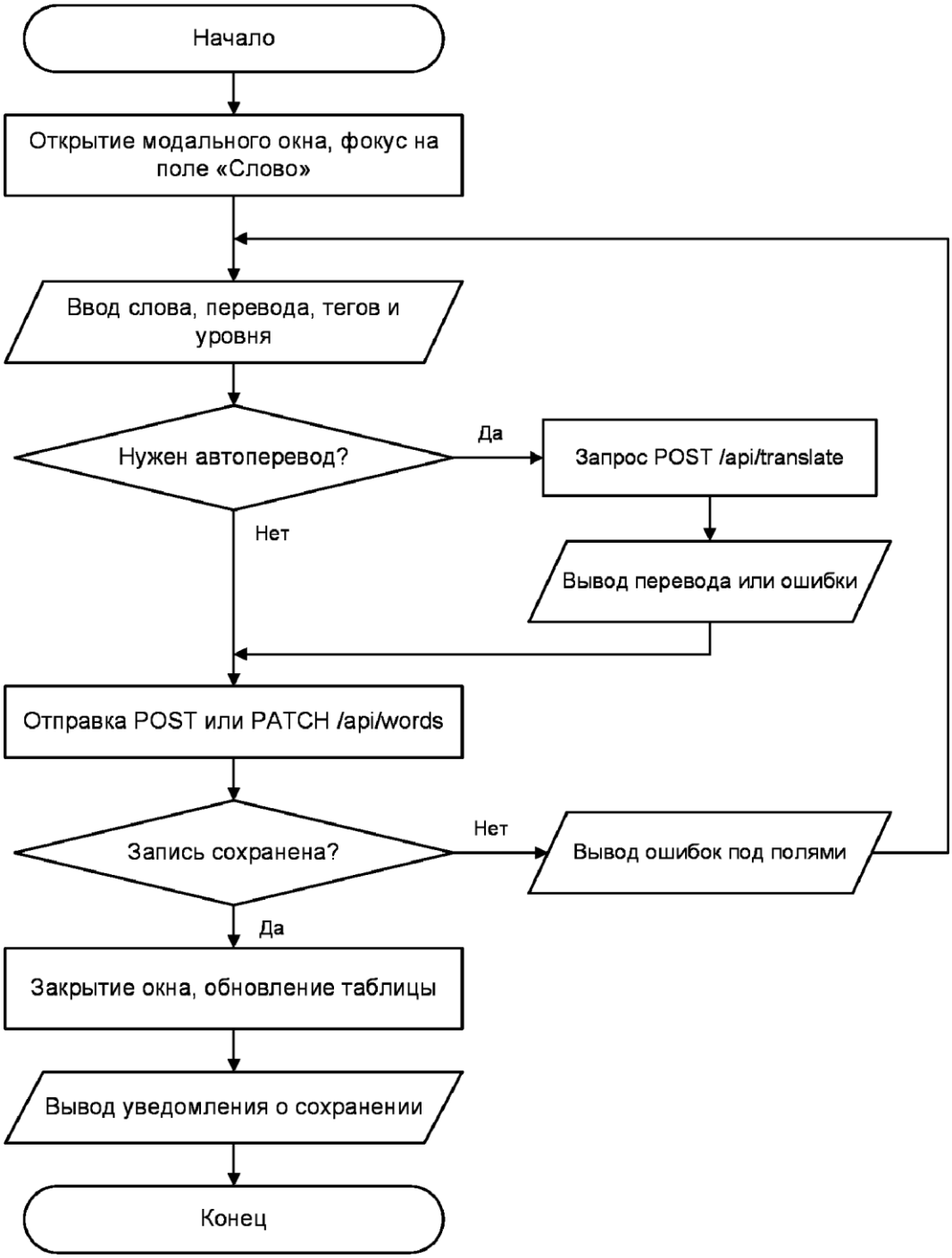


Рисунок 3.6 – Блок-схема создания словарной записи на клиенте

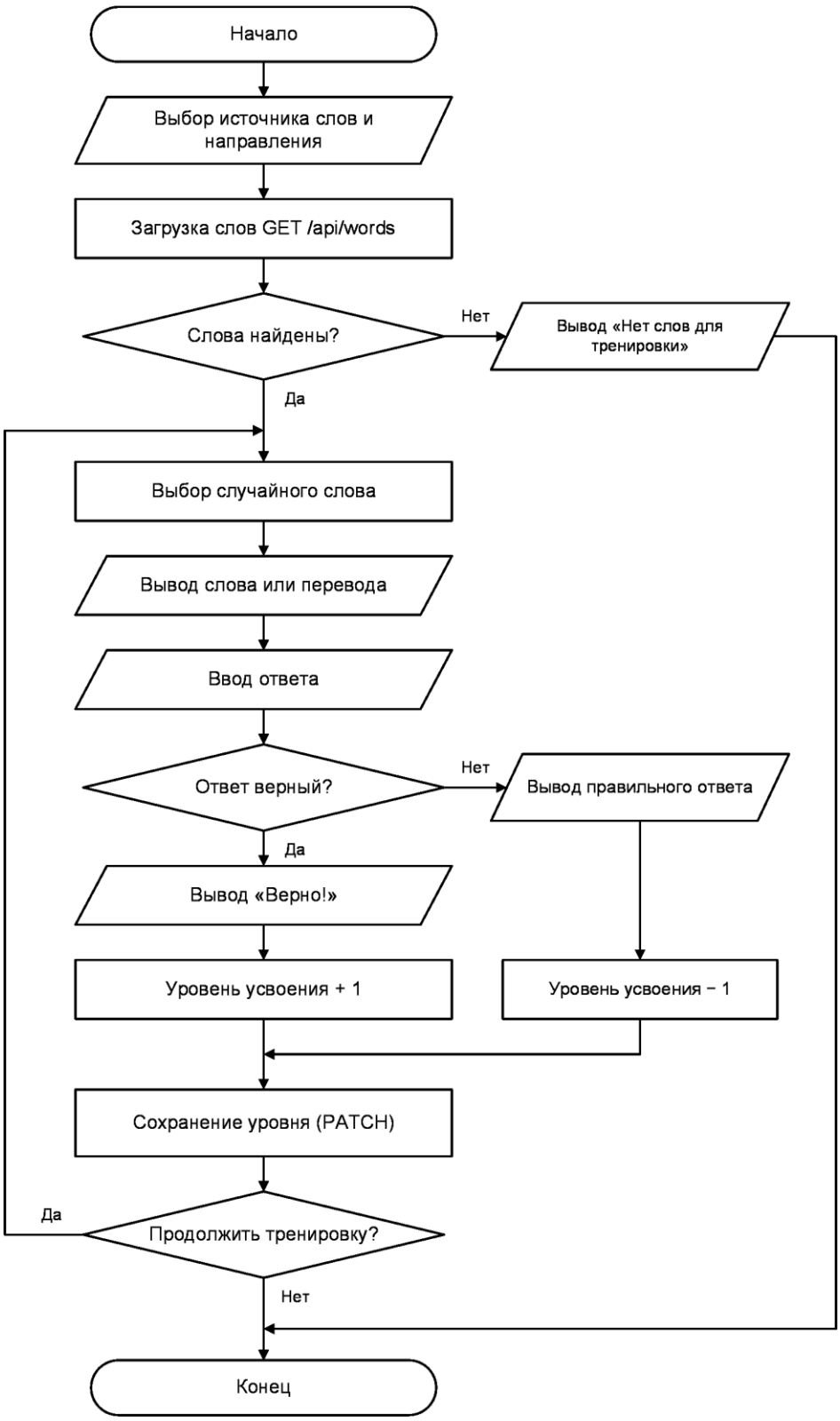


Рисунок 3.7 – Блок-схема тренировки слов на клиенте

Тренировка слов на клиенте (рисунок 3.7) начинается с выбора источника слов и направления. Если слов нет, выводится соответствующее сообщение. Иначе выбирается случайное слово из неосвоенных,

пользователь вводит ответ, программа сообщает результат, изменяет уровень усвоения и сохраняет его на сервере. Цикл повторяется, пока пользователь продолжает тренировку.

Алгоритм перевода на сервере проверяет сессию, текст и пару языков, выполняет HTTP-запрос к внешнему сервису с ограничением времени ожидания, нормализует результат и возвращает строку перевода либо код ошибки. Схема приведена на рисунке 3.8.

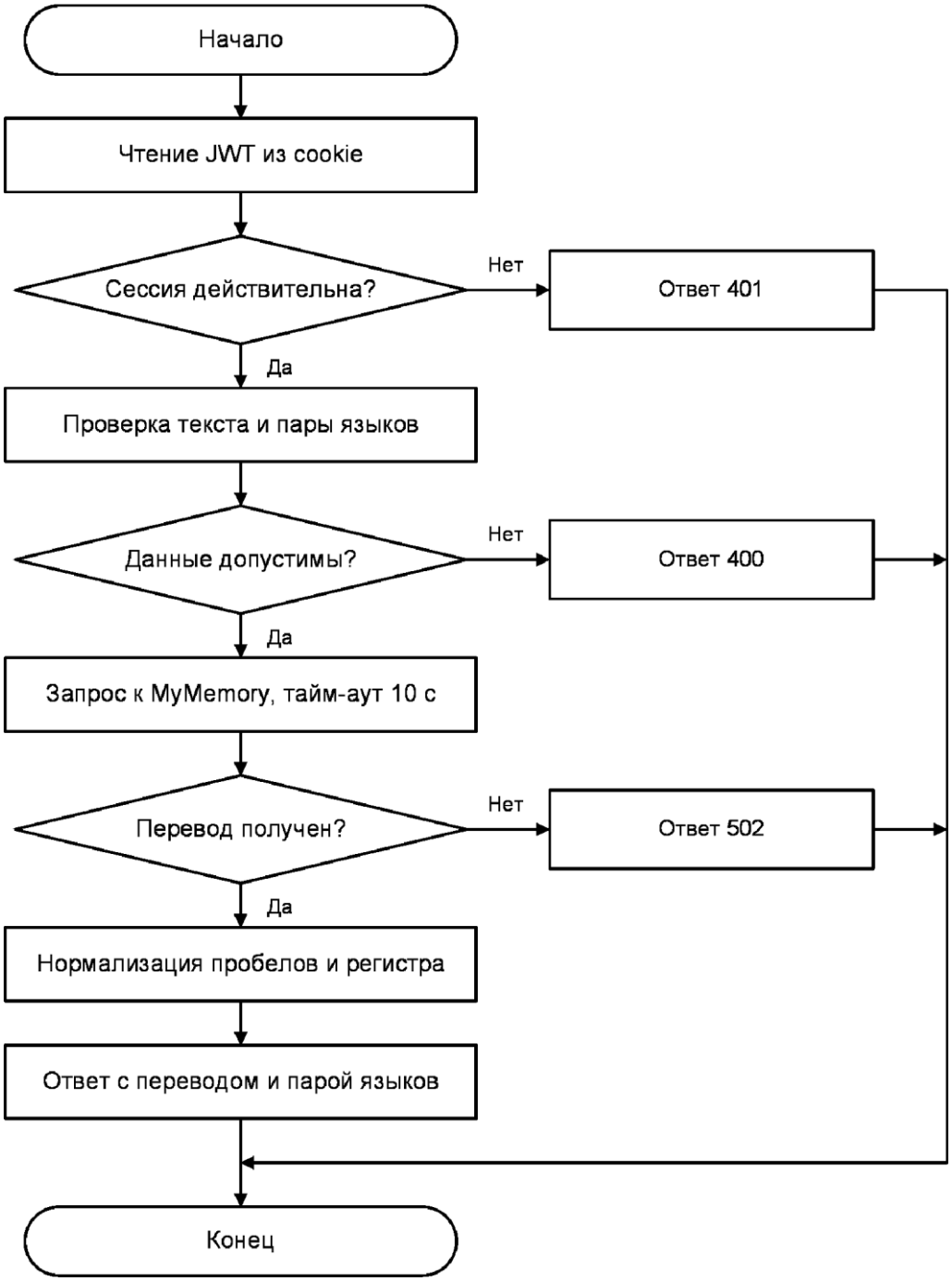


Рисунок 3.8 – Блок-схема получения перевода на сервере

Доставка уведомлений начинается с открытия потока SSE. При действительной сессии клиент регистрируется в памяти процесса и получает приветственный кадр. Пока клиент подключён, каждое новое уведомление отправляется ему в формате data: JSON; после отключения подписчик удаляется. Схема приведена на рисунке 3.9.

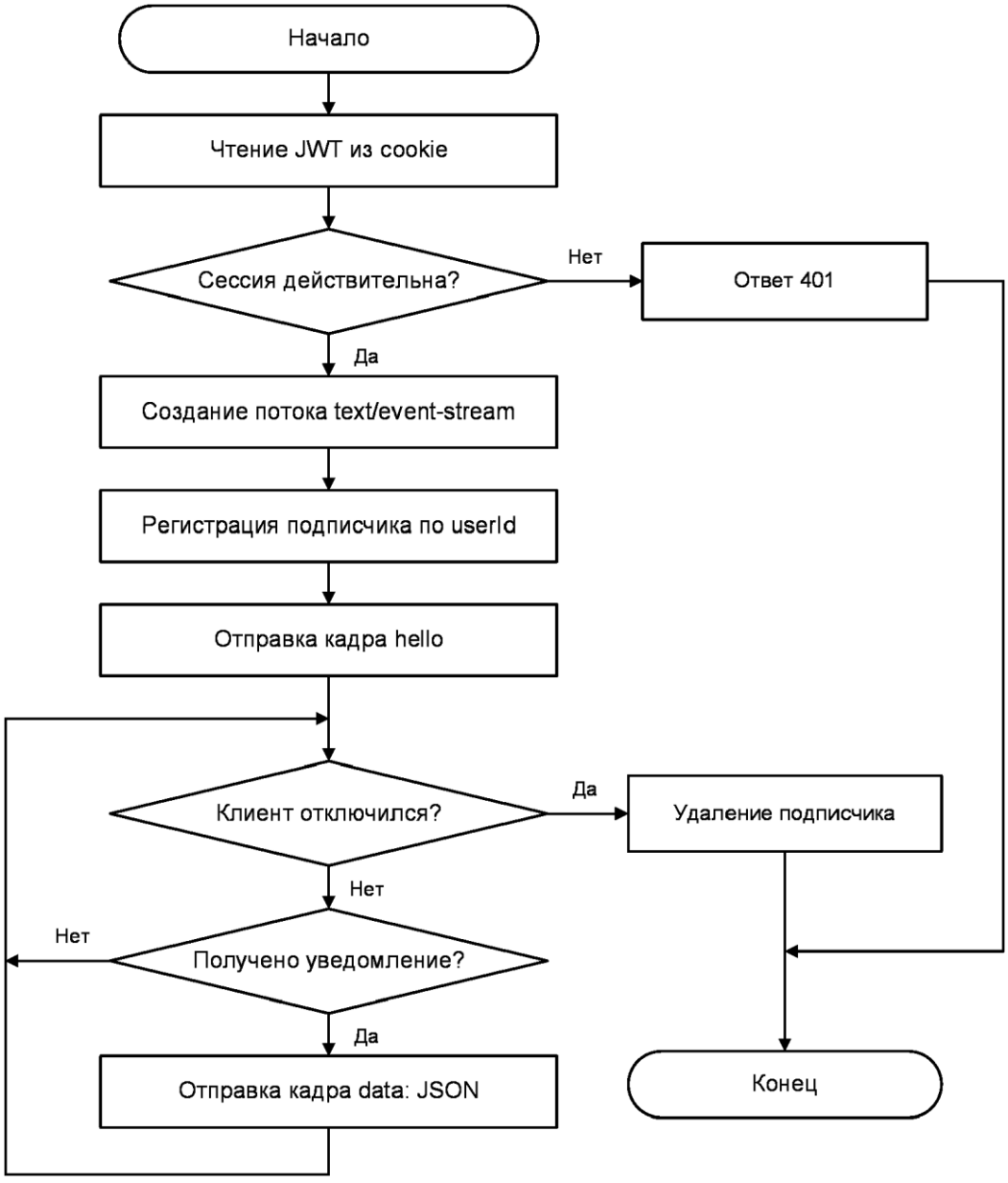


Рисунок 3.9 – Блок-схема доставки уведомлений на сервере

Обобщённый алгоритм обработки запроса к записи на сервере включает проверку сессии, разбор параметров, поиск документа по идентификатору и владельцу, выполнение операции в MongoDB, запись уведомления и формирование ответа. По этому алгоритму работают

обработчики маршрутов /api/words/[id] и /api/lessons/[id]. Схема приведена на рисунке 3.10.

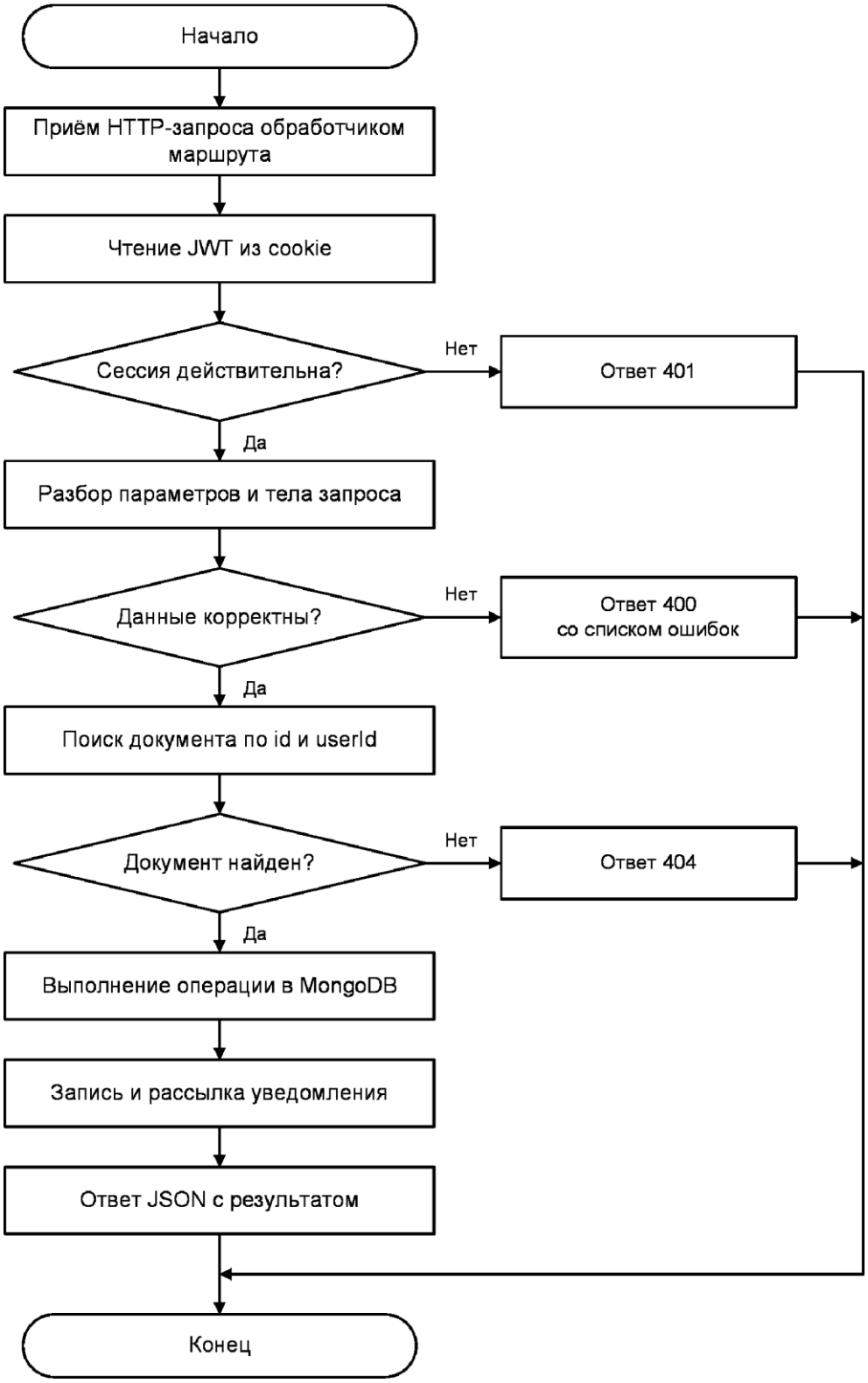


Рисунок 3.10 – Блок-схема обработки запроса к записи на сервере

4 СОЗДАНИЕ (КОНСТРУИРОВАНИЕ) ПРОГРАММНОГО СРЕДСТВА

4.1 Средства разработки

Конструирование выполнено в среде Windows с использованием Node.js, TypeScript и фреймворка Next.js 14. Серверная логика размещена в обработчиках маршрутов Route Handlers. Для доступа к MongoDB применена библиотека Mongoose. Пароли хэшируются библиотекой bcryptjs, сессии подписываются JSON Web Token [2–4]. Исходный код хранится в репозитории Git.

Контейнеризация выполнена Docker Compose. Служба mongo поднимает СУБД MongoDB 7 и хранит данные в именованном томе, служба app собирает рабочий образ приложения и запускается после службы mongo. Параметры подключения к базе, секрет подписи JWT и адрес сервиса перевода передаются через переменные окружения. Описание служб приведено в листинге А.1 приложения А.

Образ приложения собирается в три этапа. На первом этапе устанавливаются зависимости, на втором выполняется сборка Next.js, на третьем в минимальный образ Node.js 20 копируется только автономная серверная сборка и статические файлы. Процесс в контейнере запускается от непривилегированного пользователя (листинг А.2) [7].

Структура каталогов отражает разделение на страницы App Router, обработчики маршрутов в каталоге src/app/api, общие компоненты в src/components и библиотечные модули в src/lib. Подключение к базе данных выполняется функцией connectDb. Соединение кэшируется в глобальном объекте процесса, чтобы повторные запросы не создавали новых подключений. Порядок запуска служб не гарантирует, что СУБД уже принимает соединения, поэтому выполняется до 20 попыток подключения с интервалом 2 с. После первого подключения проверяется наличие демонстрационного пользователя; если его нет, создаются демонстрационные слова и уроки (листинг А.3).

4.2 Реализация аутентификации и профиля

Модуль аутентификации формирует JWT с идентификатором пользователя, записывает его в файл cookie lingua_session с флагами httpOnly и sameSite=lax и сроком действия 7 суток [8]. Функция getCurrentUser читает файл cookie, проверяет подпись токена и загружает пользователя из базы.

Если токен отсутствует, просрочен или подделан, функция возвращает пустое значение, и обработчик маршрута отвечает кодом 401. Функция publicUser формирует данные пользователя для клиента без хэша пароля (листинг А.4).

Обработчик регистрации проверяет длину имени, формат почты, длину пароля от 8 до 72 символов и принадлежность выбранных языков справочнику. Все найденные ошибки объединяются в одно сообщение. Почта приводится к нижнему регистру, при её занятости возвращается код 409. Пароль преобразуется функцией bcrypt.hash с коэффициентом сложности 10, после создания документа записывается уведомление «Регистрация завершена» и устанавливается файл cookie сессии (листинг А.5). Вход сравнивает пароль с хэшем функцией bcrypt.compare; при неверной почте или пароле возвращается одинаковое сообщение, чтобы по ответу нельзя было определить существование учётной записи.

В профиле пользователь изменяет имя, родной и изучаемый языки. Родной язык используется как язык перевода по умолчанию, изучаемый – как язык новых слов и уроков. Смена пароля выполняется отдельным маршрутом: текущий пароль сверяется с хэшем, новый пароль проверяется на длину и на несовпадение с текущим, после чего хэш перезаписывается и пользователю отправляется уведомление (листинг А.6).

4.3 Проверка данных и справочники

Все входные данные проверяются на сервере независимо от проверок в браузере. Модуль validate.ts содержит функции проверки обязательных и необязательных строк с ограничением длины, проверки почты, приведения уровня усвоения к диапазону от 0 до 5 и разбора тегов. Теги очищаются от пробелов, приводятся к нижнему регистру, обрезаются до 30 символов, повторы удаляются, число тегов ограничивается двенадцатью. Функция escapeRegex экранирует специальные символы строки поиска, поэтому ввод вида «a.b» ищется буквально. Функция parseListQuery проверяет поле сортировки по белому списку и ограничивает число записей в ответе (листинг А.7).

Модуль catalog.ts хранит справочники изучаемых и родных языков с кодами для синтеза речи, частей речи, уровней владения языком и типовых тегов (листинг А.8). Он подключается и на сервере, и на клиенте: сервер проверяет по нему присланные значения, клиент строит из него выпадающие списки. Часть речи принимается как в виде кода, так и в виде русского названия и сохраняется в виде кода.

4.4 Реализация словаря и уроков

Модели Mongoose задают структуру документов, ограничения длины полей, значения по умолчанию и индексы. Модель пользователя содержит уникальный индекс по почте (листинг А.9). Модель слова хранит слово, перевод, транскрипцию, язык, часть речи, пример, заметку, теги и уровень усвоения (листинг А.10). Модель урока хранит название, текст, язык, уровень и массив ссылок на слова (листинг А.11).

Чтение полей словарной записи вынесено в функцию readWordFields (листинг А.12). Она работает в двух режимах: при создании все обязательные поля должны присутствовать, при изменении проверяются только переданные поля. Это позволяет странице тренировки отправлять только новое значение уровня усвоения.

Коллекция слов обслуживается маршрутами GET и POST /api/words (листинг А.13). Поиск строится как логическое ИЛИ по полям слова, перевода и тегов, фильтры по тегу, языку и уроку добавляются к условию выборки. Если указан урок, сначала проверяется, что он принадлежит пользователю, а затем выбираются только слова из его списка. Новое слово получает язык из профиля, если язык не указан явно.

Маршруты GET, PATCH и DELETE /api/words/[id] ищут документ одновременно по идентификатору и по владельцу (листинг А.14). Некорректный идентификатор и чужая запись обрабатываются одинаково – ответом 404. При изменении только уровня усвоения уведомление не создаётся, чтобы тренировка не засоряла журнал. При удалении слова его идентификатор исключается из всех уроков пользователя операцией $pull.

Уроки устроены аналогично. Функция readLessonFields проверяет название, текст и уровень, а из присланного списка слов оставляет только корректные идентификаторы слов текущего пользователя (листинг А.15). Функция serializeLessons одним запросом загружает слова всех уроков списка и подставляет вместо идентификаторов слово, перевод, транскрипцию и язык.

4.5 Интеграция со сторонним сервисом перевода

Перевод выполняется обработчиком POST /api/translate (листинг А.16). После проверки сессии и текста длиной до 500 символов определяется пара языков. Если языки не переданы, используется перевод с изучаемого языка пользователя на родной. Оба языка должны входить в справочник и различаться, иначе возвращается ответ 400. Затем формируется адрес сервиса

MyMemory с параметрами q и langpair, запрос ограничивается тайм-аутом 10 с.

Из ответа внешнего сервиса извлекается поле responseData.translatedText. Сервис иногда возвращает перевод прописными буквами или с повторяющимися пробелами, поэтому результат нормализуется: пробелы схлопываются, а текст, набранный прописными буквами, приводится к регистру исходной фразы, если исходная фраза сама не набрана прописными буквами. Таким образом аббревиатуры сохраняются. Тайм-ауты, неуспешные коды и пустой результат превращаются в ответ 502 с сообщением на русском языке.

На клиенте перевод доступен в двух местах. В форме слова две кнопки переводят слово на родной язык и перевод обратно на изучаемый язык. Раздел «Переводчик» позволяет выбрать любую поддерживаемую пару языков, поменять их местами одной кнопкой, запустить перевод сочетанием клавиш Ctrl+Enter и открыть форму слова с уже заполненными полями. При обмене языков местами исходный текст и перевод тоже меняются местами.

4.6 Реализация уведомлений SSE

Каждое уведомление сначала сохраняется в коллекции notices (листинг А.17), поэтому журнал доступен и после перезагрузки страницы. Модуль sse.ts хранит для каждого пользователя множество функций отправки, так как кабинет может быть открыт в нескольких вкладках. Функция pushEvent сериализует данные в кадр вида data: JSON и отправляет его всем подключениям пользователя; отключившиеся подписчики удаляются из множества (листинг А.18). Функция notify создаёт документ уведомления и передаёт его в pushEvent (листинг А.19).

Маршрут /api/notifications/stream проверяет сессию, открывает ReadableStream с заголовком text/event-stream, регистрирует подписчика и сразу отправляет приветственный кадр. При закрытии соединения браузером вызывается функция отписки (листинг А.20) [9].

На клиенте макет кабинета загружает журнал уведомлений и открывает объект EventSource (листинг А.21). Новое событие добавляется в журнал и во всплывающее сообщение. Одновременно отображается не более трёх сообщений, каждое скрывается через 8 с либо по кнопке закрытия. Число непрочитанных уведомлений показывается рядом с заголовком журнала. Команда «Прочитать все» отправляет запрос PATCH /api/notifications, отмечает записи прочитанными и закрывает все всплывающие сообщения.

4.7 Модальные формы, выпадающие списки и озвучивание

Формы создания и редактирования слов и уроков открываются в модальном окне, поэтому список записей остаётся на экране. Компонент Modal при открытии переводит фокус на поле, отмеченное атрибутом data-autofocus, а при его отсутствии – на первое поле формы. Окно закрывается клавишей Escape, кнопкой закрытия или щелчком по затемнённому фону. На время показа окна прокрутка страницы блокируется, после закрытия фокус возвращается к элементу, который открыл окно (листинг А.22).

Форма слова реализована отдельным компонентом WordForm и используется в разделах «Слова» и «Переводчик» (листинг А.23). При редактировании форма заполняется данными выбранной записи, при сохранении из переводчика – готовыми значениями слова и перевода. Чтобы при повторном открытии не появлялись данные предыдущей записи, компонент создаётся заново при каждом открытии окна. Ошибки сервера выводятся под полями формы, окно при этом не закрывается.

Язык, часть речи и уровень урока выбираются из выпадающих списков, уровень усвоения – переключателем из шести кнопок. Компонент TagPicker показывает выбранные теги в виде меток с кнопкой удаления, предлагает выпадающий список из тегов пользователя и типовых тегов, а также позволяет ввести новый тег (листинг А.24). В форме урока слова выбираются флажками из словаря с поиском; в список попадают слова на языке урока.

Озвучивание выполняется компонентом SpeakButton через интерфейс синтеза речи браузера speechSynthesis (листинг А.25). Код языка записи преобразуется в код голоса, например en-US или de-DE, из доступных голосов выбирается голос с подходящим языком, скорость речи немного снижена для лучшего восприятия. Предыдущая фраза прерывается перед началом новой. Если браузер не поддерживает синтез речи, кнопка неактивна и показывает подсказку.

4.8 Реализация тренировки слов

Страница «Прогресс» объединяет статистику и тренировку (листинг А.26). Статистика включает число слов и уроков, число освоенных слов, распределение слов по языкам и гистограмму по уровням усвоения. Тренировка работает в двух направлениях: «слово – перевод» и «перевод – слово». Источником слов является весь словарь или выбранный урок; из окна урока можно перейти к тренировке сразу с выбранным уроком.

Очередное слово выбирается случайно среди слов с уровнем ниже пятого, при этом одно и то же слово не показывается дважды подряд. Ответ

сравнивается без учёта регистра и крайних пробелов с каждым вариантом, если в переводе их несколько через запятую или точку с запятой. Верный ответ увеличивает уровень на единицу, неверный – уменьшает, новый уровень сохраняется запросом PATCH и сразу отражается в статистике. Подсказка показывает пример употребления либо первую букву и длину слова. Счётчик верных ответов отображается в заголовке карточки тренировки.

4.9 Адаптивная вёрстка

Стили собраны в globals.css без сторонних библиотек компонентов, чтобы оформление оставалось предсказуемым в контейнере [10]. Сетки стартовой страницы, карточек и статистики переходят к одному столбцу при ширине менее 900 пикселей. Шапка на узком экране выстраивается вертикально: знак приложения, имя пользователя и кнопка выхода не накладываются друг на друга. Кнопки и поля ввода имеют высоту не менее 40 пикселей, чтобы их можно было нажать пальцем.

Таблица словаря при узкой ширине получает горизонтальную прокрутку, заголовки столбцов сохраняются. Карточки уроков занимают всю ширину колонки, длинный текст урока переносится по словам. Модальное окно на телефоне прижимается к нижнему краю экрана и занимает всю ширину, а его содержимое прокручивается внутри окна. Журнал уведомлений располагается под рабочей областью, поэтому основные действия остаются в зоне первого экрана.

5 ТЕСТИРОВАНИЕ, ПРОВЕРКА РАБОТОСПОСОБНОСТИ И АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

5.1 Виды тестирования

Проверка работоспособности включала функциональное тестирование программного интерфейса и пользовательского интерфейса, негативные сценарии проверки данных, проверку изоляции данных между учётными записями и проверку адаптивности. Тесты выполнялись на локальном контуре Docker Compose: MongoDB в отдельном контейнере, приложение на порту 3000.

Для программного интерфейса составлен сценарий на JavaScript, который выполняется в Node.js. Сценарий регистрирует тестовые учётные записи, последовательно отправляет запросы функцией fetch, сохраняет файл cookie сессии между запросами и сравнивает коды и содержимое ответов с ожидаемыми. После прогона тестовые учётные записи и их данные удаляются из базы, демонстрационные данные не затрагиваются. Пользовательский интерфейс проверялся вручную в браузере на демонстрационной учётной записи, в том числе в режиме эмуляции мобильного устройства.

5.2 Тест-кейсы

Тест-кейсы сведены в таблицу 5.1. Ожидаемый результат формулировался до выполнения теста. Тест-кейсы ТК-1 – ТК-31 относятся к программному интерфейсу, ТК-32 – ТК-42 – к пользовательскому интерфейсу. Изоляция данных проверялась второй учётной записью: обращение к чужому уроку завершается ответом 404, так как фильтр выборки включает userId (ТК-25).

Таблица 5.1 – Результаты функционального тестирования

Идентификатор	Действие	Ожидаемый результат	Факт
ТК-1	Регистрация с паролем из 4 символов	Код 400, сообщение о длине пароля	Успешно
ТК-2	Регистрация с некорректной почтой	Код 400	Успешно

Идентификатор	Действие	Ожидаемый результат	Факт
ТК-3	Регистрация с изучаемым языком вне списка	Код 400	Успешно
ТК-4	Повторная регистрация той же почты	Код 409	Успешно
ТК-5	Регистрация с корректными данными	Код 200, данные пользователя, cookie сессии	Успешно
ТК-6	Вход demo@lingua.local / Demo12345	Переход в кабинет	Успешно
ТК-7	Вход с неверным паролем	Код 401	Успешно
ТК-8	GET /api/words без cookie	Код 401	Успешно
ТК-9	Создание слова без перевода	Код 400	Успешно
ТК-10	Создание слова с частью речи «существительное» и тегами «Животные, быт, животные»	Часть речи noun, теги «животные, быт», язык из профиля	Успешно
ТК-11	Создание слова с частью речи вне списка	Код 400	Успешно
ТК-12	Запрос списка тегов	Теги без повторов по алфавиту	Успешно
ТК-13	Фильтр слов по тегу «быт»	Только слова с этим тегом	Успешно
ТК-14	Поиск по части перевода «соба»	Найдено слово Hund	Успешно
ТК-15	Поиск по строке «([»	Код 200, пустой список	Успешно
ТК-16	Сортировка по слову с ограничением в одну запись	Одна запись, первая по алфавиту	Успешно
ТК-17	Сортировка по полю passwordHash	Код 200, сортировка по дате	Успешно
ТК-18	Изменение уровня усвоения на 3	Уровень сохранён, уведомление не создано	Успешно
ТК-19	Удаление слова, входящего в урок	Слово удалено и исключено из урока	Успешно
ТК-20	Создание урока с текстом короче 10 символов	Код 400	Успешно

Идентификатор	Действие	Ожидаемый результат	Факт
ТК-21	Создание урока с двумя своими словами, чужим и некорректным идентификатором	В уроке два слова	Успешно
ТК-22	Запрос слов урока	Только слова урока	Успешно
ТК-23	Фильтр уроков по уровню B2 при отсутствии таких уроков	Пустой список	Успешно
ТК-24	Изменение уровня урока на Z9	Код 400	Успешно
ТК-25	Чтение и изменение урока другим пользователем	Код 404	Успешно
ТК-26	Перевод «собака» с русского на английский	Перевод «dog»	Успешно
ТК-27	Перевод «dog» с английского на русский	Перевод «собака»	Успешно
ТК-28	Перевод при одинаковых языках	Код 400	Успешно
ТК-29	Изменение профиля: родной язык белорусский, изучаемый немецкий	Профиль сохранён, новое слово получает немецкий язык	Успешно
ТК-30	Смена пароля с неверным текущим паролем	Код 400 «Текущий пароль указан неверно»	Успешно
ТК-31	Смена пароля с верным текущим паролем	Вход со старым паролем – 401, с новым – 200	Успешно
ТК-32	Нажатие «+ Добавить слово»	Модальное окно, фокус в поле «Слово», список виден	Успешно
ТК-33	Ввод слова и нажатие кнопки перевода «английский → русский»	Поле перевода заполнено	Успешно
ТК-34	Нажатие Escape в открытом окне	Окно закрывается	Успешно
ТК-35	Сохранение нового слова	Окно закрыто, запись подсвечена, всплывающее уведомление	Успешно

Идентификатор	Действие	Ожидаемый результат	Факт
ТК-36	Удаление слова с подтверждением	Запись исчезает из таблицы	Успешно
ТК-37	Перевод в разделе «Переводчик» и нажатие «В словарь»	Форма слова с заполненными полями	Успешно
ТК-38	Кнопка «Тренировать слова урока» в окне урока	Тренировка по словам урока	Успешно
ТК-39	Верный ответ в направлении «перевод – слово»	Сообщение «Верно!», уровень увеличен	Успешно
ТК-40	Ответ, совпадающий с одним из вариантов перевода через запятую	Ответ засчитан	Успешно
ТК-41	Нажатие «Прочитать все»	Счётчик сброшен, всплывающие сообщения закрыты	Успешно
ТК-42	Открытие кабинета на ширине 390 пикселей	Блоки в один столбец, окно формы у нижнего края	Успешно

5.3 Анализ результатов

Все тест-кейсы выполнены успешно. Тестирование подтвердило выполнение требований задания: программный интерфейс REST функционирует, нереляционная база используется для всех сущностей, реализованы вход и регистрация, создание, просмотр, изменение и удаление записей, поиск, фильтрация и сортировка, проверка данных и шифрование паролей, адаптивная вёрстка, двусторонний перевод через MyMemory, тренировка слов и уведомления SSE.

Ограничением учебной реализации является хранение подписчиков SSE в памяти процесса: при нескольких экземплярах приложения потребуется внешняя шина сообщений. Для курсового контура с одним контейнером app ограничение приемлемо. Качество перевода и наличие голосов для озвучивания зависят от внешнего сервиса и от браузера пользователя.

6 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ

6.1 Установка и запуск

Для запуска необходимы Docker и Docker Compose. Исходные файлы размещаются в каталоге lang-app. В корне каталога находятся Dockerfile и docker-compose.yml. Запуск выполняется командой

```
docker compose up --build
```

После сборки образа приложение доступно по адресу http://localhost:3000. Служба MongoDB не публикуется наружу и доступна приложению по имени хоста mongo. Переменные MONGODB_URI, JWT_SECRET, JWT_EXPIRES и TRANSLATE_ENDPOINT задаются в docker-compose.yml. Для работы перевода серверу нужен доступ в интернет. Стартовая страница приведена на рисунке 6.1.

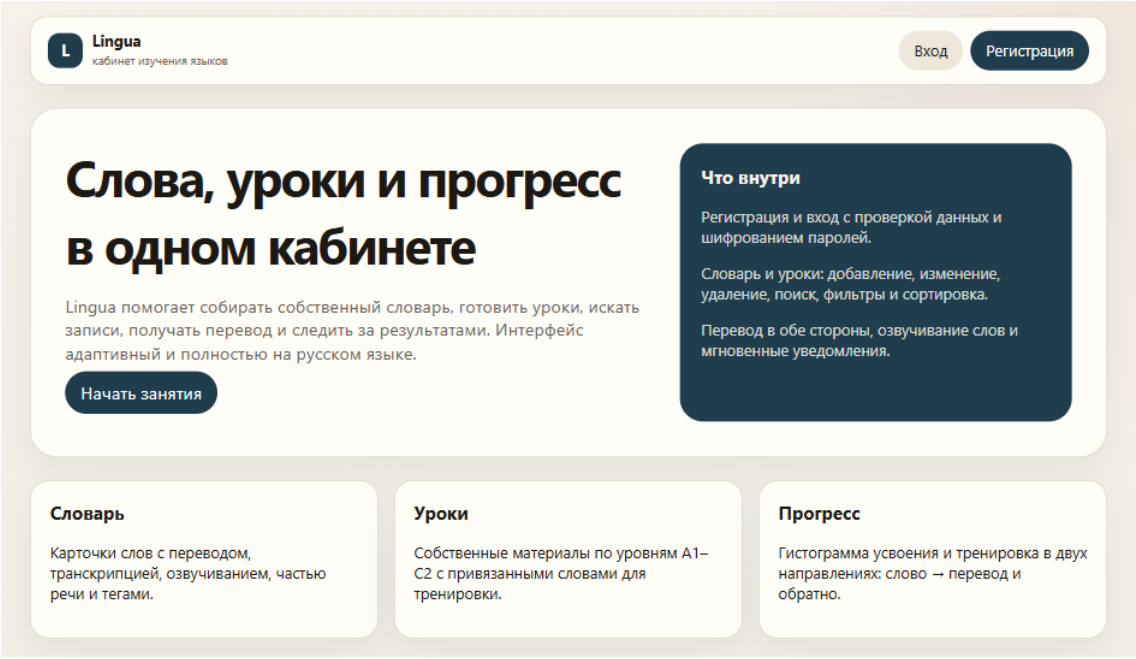


Рисунок 6.1 – Стартовая страница приложения Lingua

При первом старте создаётся демонстрационная учётная запись со словами и уроками:

– почта: demo@lingua.local;

– пароль: Demo12345.

Остановка выполняется сочетанием Ctrl+C в том же терминале либо командой docker compose down. Данные MongoDB сохраняются в томе lingua_mongo и не теряются при перезапуске.

6.2 Руководство пользователя

6.2.1 Регистрация и вход. На стартовой странице выбирается «Регистрация». Форма регистрации показана на рисунке 6.2. Вводятся имя, почта и пароль не короче восьми символов, из выпадающих списков выбираются родной и изучаемый языки. При ошибке сообщение выводится под формой, при успешной регистрации сразу открывается личный кабинет.

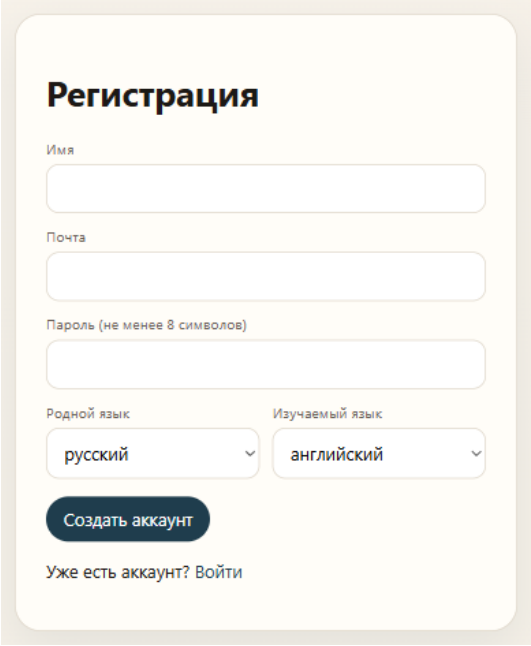


Рисунок 6.2 – Форма регистрации

Для входа в существующую учётную запись используется ссылка «Войти». Форма входа показана на рисунке 6.3. Вводятся почта и пароль, при неверных данных выводится сообщение «Неверная почта или пароль».



Рисунок 6.3 – Форма входа

6.2.2 Обзор. После входа открывается раздел «Обзор» (рисунок 6.4). В верхней части кабинета находятся имя пользователя, кнопка «Выйти» и навигация по разделам. Раздел показывает число слов, уроков, освоенных слов и изучаемых языков, пять недавно добавленных слов и пять слов, которые требуют повторения. Кнопка «+ Добавить слово» сразу открывает форму нового слова, кнопка «Повторить слова» – тренировку. Рядом с каждым словом расположена кнопка озвучивания.

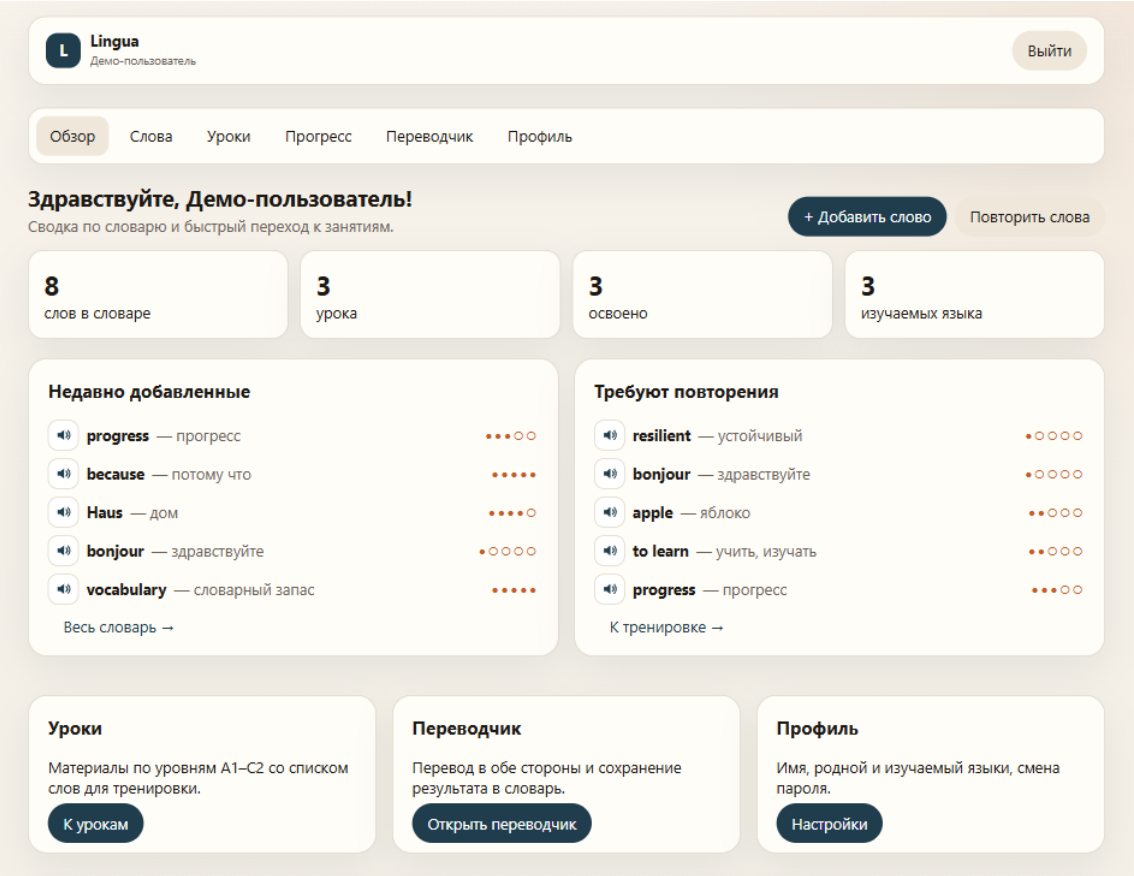


Рисунок 6.4 – Раздел «Обзор»

6.2.3 Работа со словарём. Раздел «Слова» показывает таблицу словаря (рисунок 6.5). Для каждого слова выводятся транскрипция, часть речи, перевод, пример, заметка, теги, язык и уровень усвоения в виде шкалы из пяти точек. Строка поиска находит слова по части слова, перевода или тега. Выпадающие списки над таблицей фильтруют слова по тегу и языку и задают поле и направление сортировки. Щелчок по тегу в таблице включает фильтр по этому тегу. Если по условиям ничего не найдено, выводится кнопка «Сбросить фильтры».

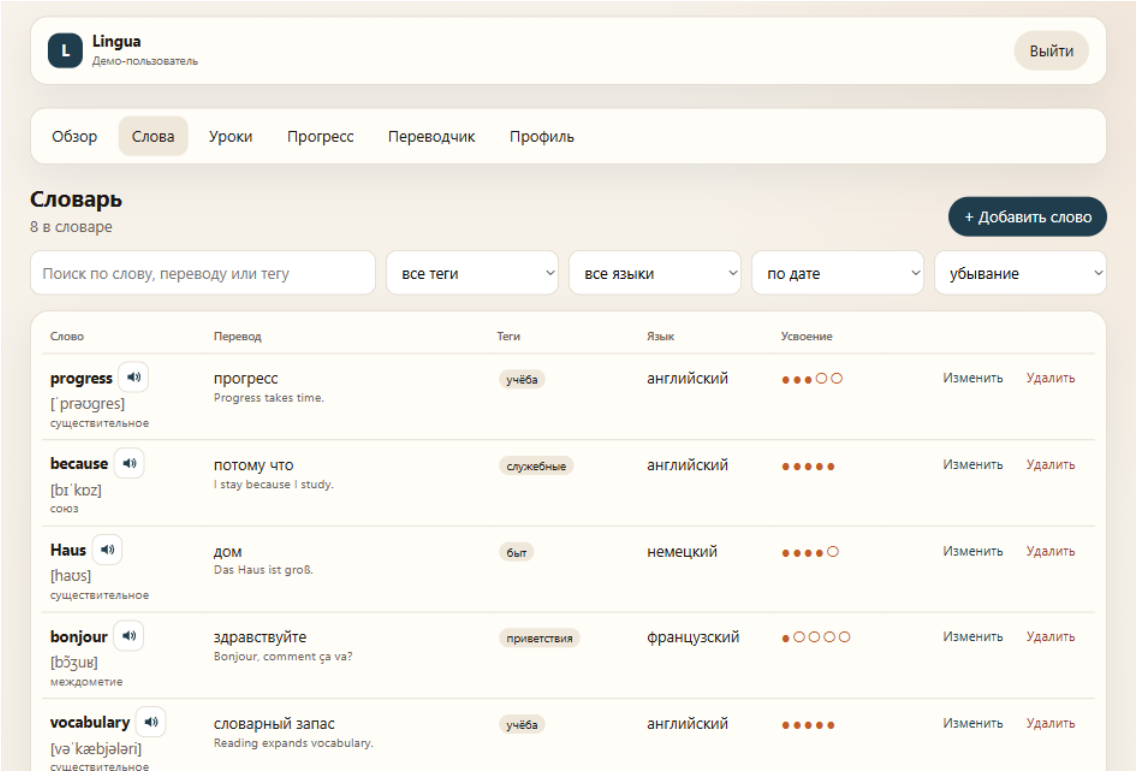


Рисунок 6.5 – Раздел «Слова»

Кнопка «+ Добавить слово» открывает форму в модальном окне (рисунок 6.6), курсор сразу находится в поле «Слово». Язык слова и часть речи выбираются из выпадающих списков. Кнопки «английский → русский» и «русский → английский» переводят слово или перевод через сервис MyMemory, полученный результат можно исправить вручную. Кнопка с динамиком рядом с полем «Слово» произносит слово. Уровень усвоения задаётся кнопками от 0 до 5. Тег выбирается из списка «+ выбрать тег» либо вводится в поле ниже с подтверждением клавишей Enter; лишний тег удаляется крестиком. Кнопка «Добавить слово» сохраняет запись, после чего окно закрывается, а новая строка подсвечивается в таблице. Если данные не прошли проверку, сообщение выводится в окне, введённые значения сохраняются.

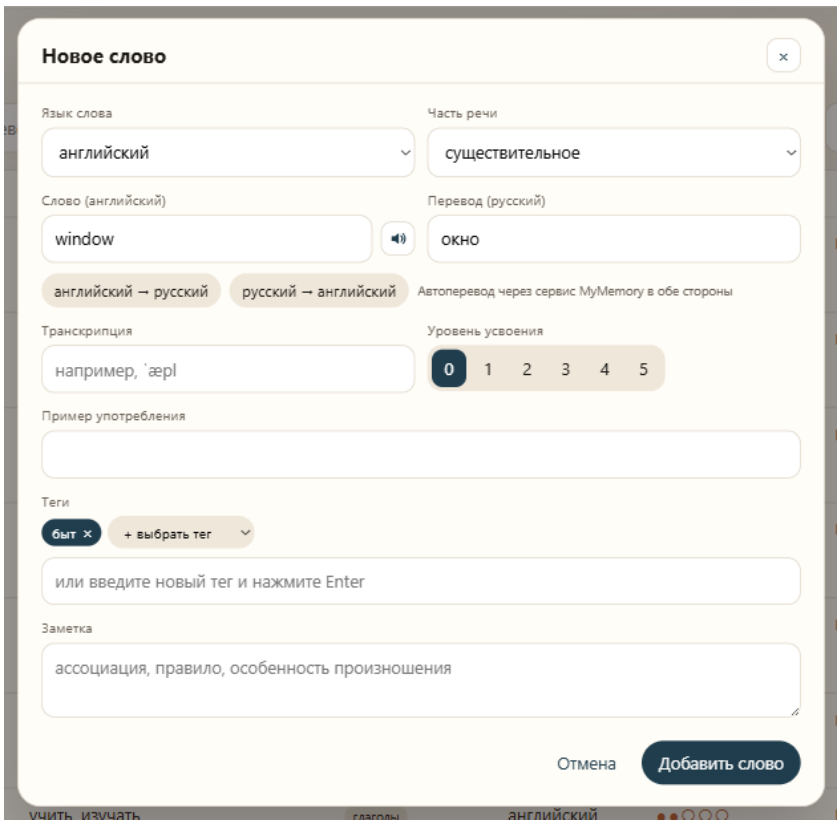


Рисунок 6.6 – Форма слова

Команда «Изменить» в строке таблицы открывает ту же форму с данными выбранного слова. Команда «Удалить» открывает окно подтверждения; после подтверждения слово удаляется из словаря и из всех уроков. Любое окно закрывается клавишей Escape, крестиком в заголовке или щелчком по затемнённому фону.

6.2.4 Работа с уроками. Раздел «Уроки» показывает карточки уроков (рисунок 6.7) с уровнем, языком, числом слов, началом текста и первыми словами урока. Уроки ищутся по названию и тексту, фильтруются по уровню и языку и сортируются по дате, названию, уровню или языку.

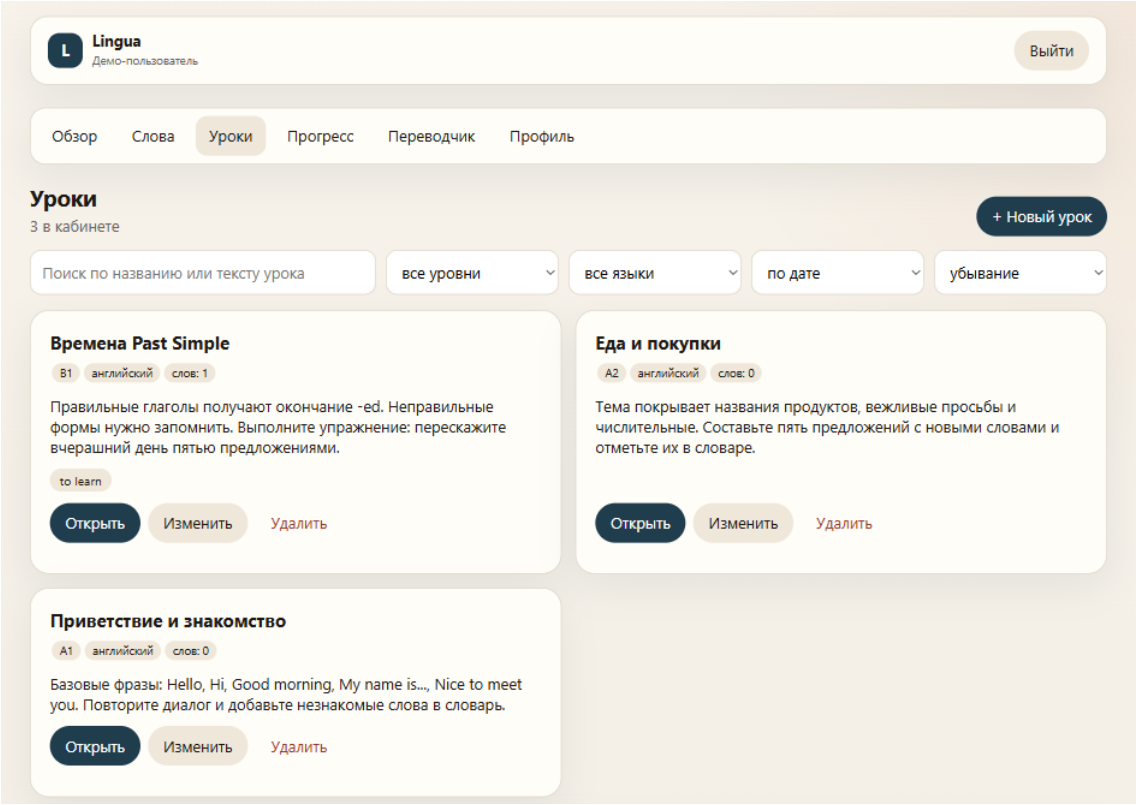


Рисунок 6.7 – Раздел «Уроки»

Кнопка «+ Новый урок» и команда «Изменить» открывают форму урока (рисунок 6.8). Вводятся название и текст урока, из выпадающих списков выбираются уровень A1–C2 и язык. Под текстом показано число введённых символов. В нижней части формы флажками отмечаются слова урока; список содержит слова словаря на языке урока, строка над списком ищет слово по написанию или переводу.

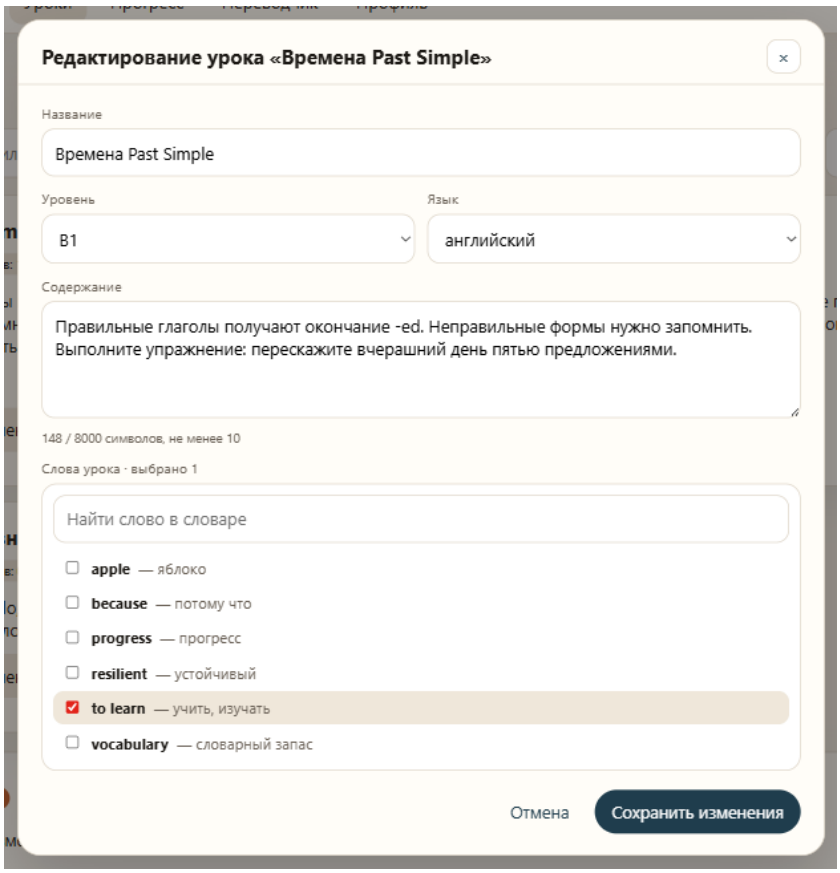


Рисунок 6.8 – Форма урока

Команда «Открыть» показывает урок для чтения (рисунок 6.9): полный текст и список слов с транскрипцией, переводом и кнопками озвучивания. Кнопка «Тренировать слова урока» открывает тренировку только по словам этого урока. Удаление урока выполняется после подтверждения, слова в словаре при этом сохраняются.



Рисунок 6.9 – Окно чтения урока

6.2.5 Прогресс и тренировка. Раздел «Прогресс» (рисунок 6.10) показывает общее число слов, число освоенных слов с уровнем 4 или 5, число уроков, распределение слов по языкам и гистограмму числа слов на каждом уровне усвоения.

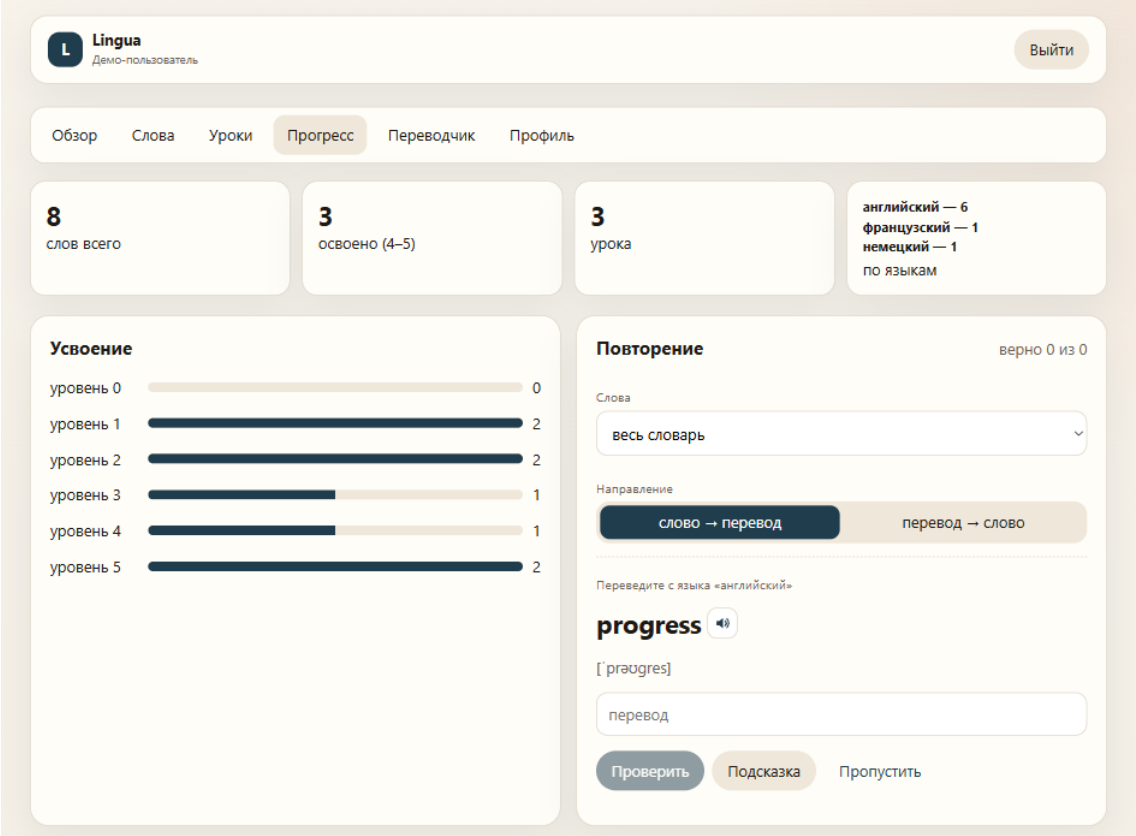


Рисунок 6.10 – Раздел «Прогресс»

В карточке «Повторение» выбираются источник слов (весь словарь или один из уроков) и направление тренировки: «слово → перевод» или «перевод → слово». Программа показывает слово или перевод, пользователь вводит ответ и нажимает «Проверить» или клавишу Enter. Если перевод содержит несколько вариантов через запятую, засчитывается любой из них. При верном ответе выводится «Верно!» и уровень усвоения увеличивается, при неверном – выводится правильный ответ и уровень уменьшается. Кнопка «Подсказка» показывает пример употребления или первую букву и длину слова, «Пропустить» переходит к другому слову без изменения уровня, «Следующее» – к следующему слову после проверки. Счётчик в заголовке карточки показывает число верных ответов.

6.2.6 Переводчик. Раздел «Переводчик» (рисунок 6.11) переводит слова и фразы длиной до 500 символов. Языки выбираются из выпадающих списков, кнопка «⇄» меняет их местами вместе с текстом и переводом.

Перевод запускается кнопкой «Перевести» или сочетанием Ctrl+Enter. Кнопки с динамиком под полями произносят исходный текст и перевод. Если одна сторона пары – изучаемый язык, а другая – родной, кнопка «В словарь» открывает форму слова с уже заполненными полями.

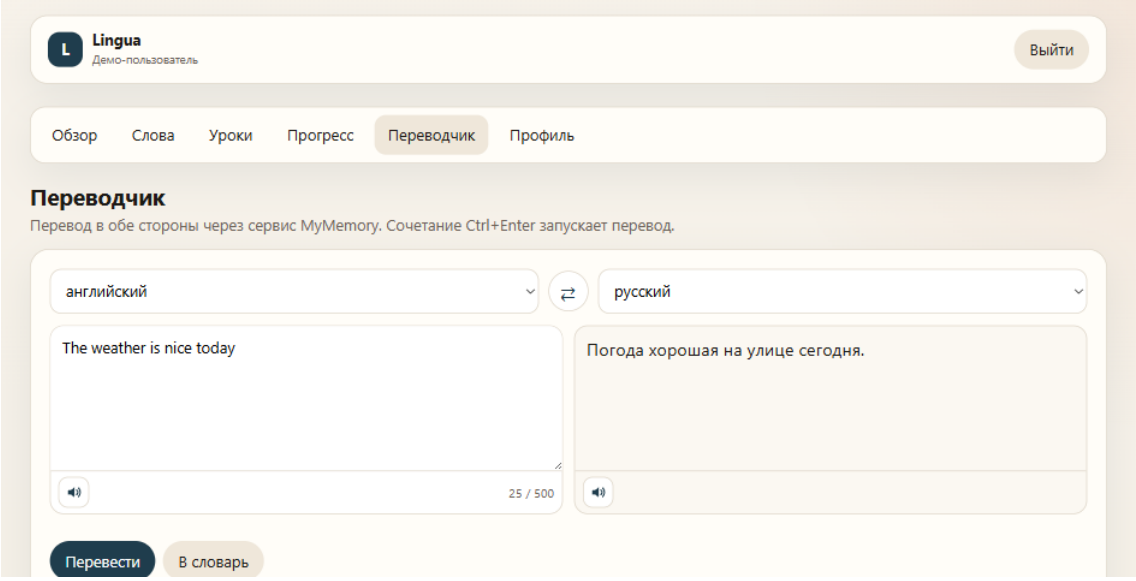


Рисунок 6.11 – Раздел «Переводчик»

6.2.7 Профиль. В разделе «Профиль» (рисунок 6.12) изменяются имя, родной и изучаемый языки. Родной язык используется как язык перевода по умолчанию, изучаемый – как язык новых слов и уроков. Для смены пароля вводятся текущий пароль и дважды новый пароль. Если текущий пароль неверен или новые пароли не совпадают, выводится сообщение об ошибке.

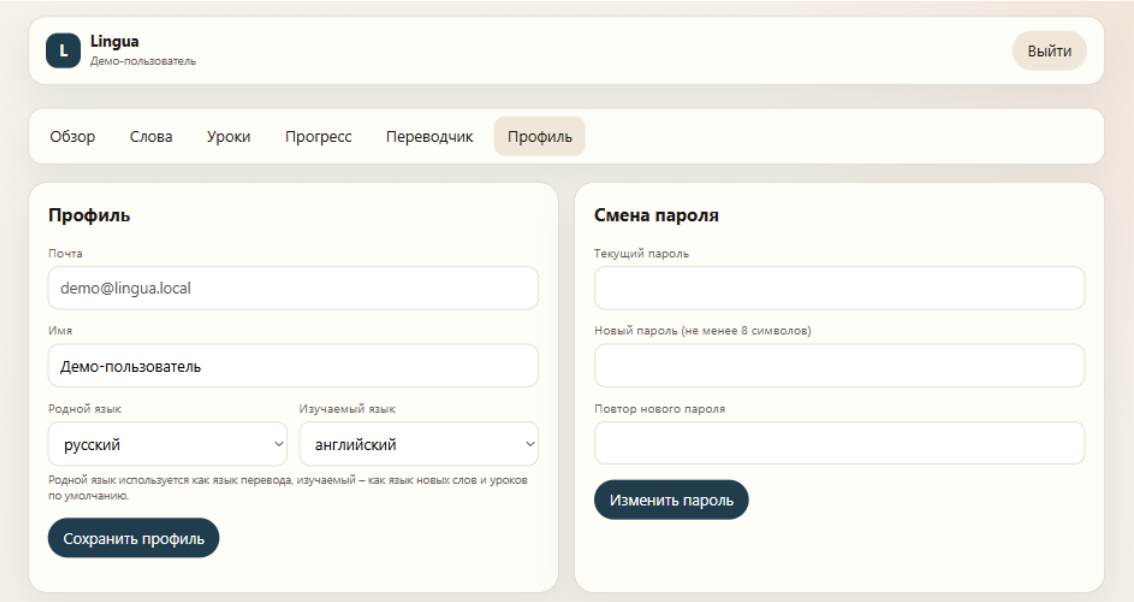


Рисунок 6.12 – Раздел «Профиль»

6.2.8 Уведомления. Журнал в нижней части кабинета показывает последние события: регистрацию, создание, изменение и удаление слов и уроков, изменение профиля и пароля. Число непрочитанных событий выводится рядом с заголовком журнала. Новое событие также появляется во всплывающем сообщении в углу экрана; сообщение исчезает через 8 с или по крестику. Кнопка «Прочитать все» отмечает все события прочитанными и закрывает всплывающие сообщения. Поток уведомлений открывается автоматически при входе в кабинет.

6.2.9 Завершение работы. Кнопка «Выйти» удаляет файл cookie сессии и возвращает на стартовую страницу.

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта разработано веб-приложение Lingua для изучения иностранных языков. Проанализированы предметная область и аналоги Duolingo, Lingualeo, Quizlet и Anki, сформулированы функциональные и нефункциональные требования, построены диаграмма вариантов использования и диаграмма развёртывания, спроектированы архитектура, документная база данных, программный интерфейс REST, пользовательский интерфейс кабинета и алгоритмы основных сценариев.

Реализованы регистрация и вход с проверкой данных и хэшированием паролей bcrypt, создание, просмотр, изменение и удаление слов и уроков, привязка слов к урокам, поиск, фильтрация и сортировка записей, личный кабинет с разделами обзора, словаря, уроков, прогресса, переводчика и профиля. Формы открываются в модальных окнах с переводом фокуса на первое поле, значения с ограниченным набором вариантов выбираются из выпадающих списков. Добавлены тренировка слов в двух направлениях по всему словарю или по словам урока, двусторонний перевод через сторонний сервис MyMemory, озвучивание слов средствами браузера, изменение профиля и смена пароля, уведомления по SSE с журналом и отметкой о прочтении, адаптивная вёрстка. Приложение упаковано в Docker Compose и запускается одной командой.

Тестирование по 42 тест-кейсам подтвердило работоспособность основных сценариев, включая негативные случаи проверки данных и изоляцию данных пользователей. Графический материал содержит схему программы на листе формата А1. Дальнейшее развитие может включать интервальное повторение по расписанию, импорт словарей и горизонтальное масштабирование канала уведомлений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Сысоев, П. В. Информационные и коммуникационные технологии в обучении иностранному языку : теория и практика / П. В. Сысоев. – М. : Глосса-Пресс, 2012. – 238 с.

[2] Next.js Documentation [Электронный ресурс]. – Режим доступа : https://nextjs.org/docs.

[3] MongoDB Manual [Электронный ресурс]. – Режим доступа : https://www.mongodb.com/docs/manual/.

[4] bcryptjs [Электронный ресурс]. – Режим доступа : https://www.npmjs.com/package/bcryptjs.

[5] MyMemory translated.net API [Электронный ресурс]. – Режим доступа : https://mymemory.translated.net/doc/spec.php.

[6] ГОСТ 19.701–90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. – Введ. 01.01.92. – М. : Изд-во стандартов, 1991. – 21 с.

[7] Docker Compose [Электронный ресурс]. – Режим доступа : https://docs.docker.com/compose/.

[8] Jones, M. JSON Web Token (JWT) : RFC 7519 / M. Jones, J. Bradley, N. Sakimura. – IETF, 2015.

[9] HTML Living Standard. Server-sent events [Электронный ресурс]. – Режим доступа : https://html.spec.whatwg.org/multipage/server-sent-events.html.

[10] Фримен, Э. Изучаем программирование на JavaScript / Э. Фримен, Э. Робсон. – СПб. : Питер, 2015. – 640 с.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинги программного средства

В приложении А приведены листинги конфигурации развёртывания, серверных модулей, моделей данных, обработчиков маршрутов и основных клиентских компонентов. Описание приведённого кода дано в разделе 4.

Листинг А.1 – Файл docker-compose.yml

```
services:
  mongo:
    image: mongo:7
    restart: unless-stopped
    volumes:
      - lingua_mongo:/data/db
  app:
    build: .
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      MONGODB_URI: mongodb://mongo:27017/lingua
      JWT_SECRET: lingua-course-work-2026
      JWT_EXPIRES: 7d
      TRANSLATE_ENDPOINT: https://api.mymemory.translated.net/get
    depends_on:
      - mongo

volumes:
  lingua_mongo:
```

Листинг А.2 – Файл Dockerfile

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm install

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV NEXT_TELEMETRY_DISABLED=1
ENV PORT=3000
ENV HOSTNAME=0.0.0.0

RUN addgroup --system --gid 1001 nodejs \
  && adduser --system --uid 1001 nextjs
```

```
COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

Листинг А.3 – Подключение к базе данных

```
import mongoose from "mongoose";
import { seedIfEmpty } from "./seed";

const MONGODB_URI = process.env.MONGODB_URI ||
"mongodb://127.0.0.1:27017/lingua";

type Cache = {
  conn: typeof mongoose | null;
  promise: Promise<typeof mongoose> | null;
  seeded: boolean;
};

const globalWithCache = global as typeof globalThis & { _linguaMongo?: Cache
};

const cache: Cache = globalWithCache._linguaMongo || {
  conn: null,
  promise: null,
  seeded: false,
};

globalWithCache._linguaMongo = cache;

export async function connectDb() {
  if (cache.conn) {
    if (!cache.seeded) {
      await seedIfEmpty();
      cache.seeded = true;
    }
    return cache.conn;
  }

  if (!cache.promise) {
    cache.promise = (async () => {
      let lastError: unknown;
      for (let attempt = 1; attempt <= 20; attempt += 1) {
        try {
          return await mongoose.connect(MONGODB_URI, {
            serverSelectionTimeoutMS: 5000,
          });
        } catch (error) {
          lastError = error;
          await new Promise((resolve) => setTimeout(resolve, 2000));
        }
      }
      throw lastError;
    })();
  }

  cache.conn = await cache.promise;
```

```
  if (!cache.seeded) {
    await seedIfEmpty();
    cache.seeded = true;
  }
  return cache.conn;
}
```

Листинг А.4 – Модуль аутентификации

```
import jwt from "jsonwebtoken";
import { cookies } from "next/headers";
import { NextResponse } from "next/server";
import { connectDb } from "./db";
import User from "./models/User";

const COOKIE = "lingua_session";

function secret() {
  return process.env.JWT_SECRET || "lingua-course-work-2026";
}

export function signToken(userId: string) {
  return jwt.sign({ sub: userId }, secret(), {
    expiresIn: (process.env.JWT_EXPIRES || "7d") as
jwt.SignOptions["expiresIn"],
  });
}

export function setSessionCookie(response: NextResponse, token: string) {
  response.cookies.set(COOKIE, token, {
    httpOnly: true,
    sameSite: "lax",
    path: "/",
    maxAge: 60 * 60 * 24 * 7,
  });
  return response;
}

export function clearSessionCookie(response: NextResponse) {
  response.cookies.set(COOKIE, "", { httpOnly: true, path: "/", maxAge: 0 });
  return response;
}

export async function getCurrentUser() {
  await connectDb();
  const token = cookies().get(COOKIE)?.value;
  if (!token) {
    return null;
  }
  try {
    const payload = jwt.verify(token, secret()) as { sub: string };
    const user = await User.findById(payload.sub);
    return user;
  } catch {
    return null;
  }
}

export function unauthorized() {
```

```
  return NextResponse.json({ error: "Требуется авторизация" }, { status: 401
});
}

export function publicUser(user: { _id: unknown; name: string; email: string;
nativeLanguage: string; targetLanguage: string }) {
  return {
    id: String(user._id),
    name: user.name,
    email: user.email,
    nativeLanguage: user.nativeLanguage,
    targetLanguage: user.targetLanguage,
  };
}
```

Листинг А.5 – Обработчик регистрации

```
import bcrypt from "bcryptjs";
import { NextRequest, NextResponse } from "next/server";
import { connectDb } from "@/lib/db";
import User from "@/lib/models/User";
import { isNativeLanguage, isStudyLanguage } from "@/lib/catalog";
import { isEmail, requireString } from "@/lib/validate";
import { publicUser, setSessionCookie, signToken } from "@/lib/auth";
import { notify } from "@/lib/notify";

export const dynamic = "force-dynamic";

export async function POST(request: NextRequest) {
  await connectDb();
  const body = await request.json().catch(() => ({}));
  const errors = [
    requireString(body.name, "Имя", 2, 80),
    requireString(body.email, "Почта", 5, 120),
    requireString(body.password, "Пароль", 8, 72),
  ].filter(Boolean) as string[];

  if (body.email && !isEmail(String(body.email))) {
    errors.push("Почта: некорректный формат");
  }
  const nativeLanguage = String(body.nativeLanguage || "ru");
  const targetLanguage = String(body.targetLanguage || "en");
  if (!isNativeLanguage(nativeLanguage)) {
    errors.push("Родной язык: выберите значение из списка");
  }
  if (!isStudyLanguage(targetLanguage)) {
    errors.push("Изучаемый язык: выберите значение из списка");
  }
  if (errors.length) {
    return NextResponse.json({ error: errors.join("; ") }, { status: 400 });
  }

  const email = String(body.email).trim().toLowerCase();
  const exists = await User.findOne({ email });
  if (exists) {
    return NextResponse.json({ error: "Пользователь с такой почтой уже
существует" }, { status: 409 });
  }

  const user = await User.create({
```

```
    name: String(body.name).trim(),
    email,
    passwordHash: await bcrypt.hash(String(body.password), 10),
    nativeLanguage,
    targetLanguage,
  });

  await notify(String(user._id), "Регистрация завершена", "Личный кабинет
готов к работе со словарём и уроками.", "success");
  const response = NextResponse.json({ user: publicUser(user) });
  return setSessionCookie(response, signToken(String(user._id)));
}
```

Листинг А.6 – Обработчик смены пароля

```
import bcrypt from "bcryptjs";
import { NextRequest, NextResponse } from "next/server";
import { getCurrentUser, unauthorized } from "@/lib/auth";
import { requireString } from "@/lib/validate";
import { notify } from "@/lib/notify";

export const dynamic = "force-dynamic";

export async function POST(request: NextRequest) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const body = await request.json().catch(() => ({}));
  const errors = [
    requireString(body.currentPassword, "Текущий пароль", 1, 72),
    requireString(body.newPassword, "Новый пароль", 8, 72),
  ].filter(Boolean) as string[];
  if (errors.length) {
    return NextResponse.json({ error: errors.join("; ") }, { status: 400 });
  }
  if (!(await bcrypt.compare(String(body.currentPassword),
user.passwordHash))) {
    return NextResponse.json({ error: "Текущий пароль указан неверно" }, {
status: 400 });
  }
  if (body.currentPassword === body.newPassword) {
    return NextResponse.json({ error: "Новый пароль должен отличаться от
текущего" }, { status: 400 });
  }
  user.passwordHash = await bcrypt.hash(String(body.newPassword), 10);
  await user.save();
  await notify(String(user._id), "Пароль изменён", "Для следующего входа
используйте новый пароль.", "warning");
  return NextResponse.json({ ok: true });
}
```

Листинг А.7 – Проверка входных данных

```
import mongoose from "mongoose";

export function isEmail(value: string) {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value);
}
```

```
export function requireString(value: unknown, field: string, min = 1, max =
500) {
  if (typeof value !== "string") {
    return `${field}: значение должно быть строкой`;
  }
  const trimmed = value.trim();
  if (trimmed.length < min) {
    return `${field}: минимум ${min} символ(ов)`;
  }
  if (trimmed.length > max) {
    return `${field}: максимум ${max} символов`;
  }
  return null;
}

export function optionalString(value: unknown, field: string, max: number) {
  if (value === undefined || value === null) {
    return null;
  }
  return requireString(String(value), field, 0, max);
}

export function clampMastery(value: unknown) {
  const n = Number(value);
  if (!Number.isFinite(n)) {
    return 0;
  }
  return Math.max(0, Math.min(5, Math.round(n)));
}

export function escapeRegex(value: string) {
  return value.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
}

export function parseTags(value: unknown) {
  const list = Array.isArray(value) ? value : String(value || "").split(",");
  const seen = new Set<string>();
  const tags: string[] = [];
  for (const item of list) {
    const tag = String(item).trim().toLowerCase().slice(0, 30);
    if (tag && !seen.has(tag)) {
      seen.add(tag);
      tags.push(tag);
    }
  }
  return tags.slice(0, 12);
}

export function parseObjectIds(value: unknown) {
  if (!Array.isArray(value)) {
    return [];
  }
  return Array.from(new Set(value.map(String))).filter((id) =>
mongoose.Types.ObjectId.isValid(id));
}

const SORT_WORDS = new Set(["term", "translation", "createdAt", "updatedAt",
"mastery", "language"]);
const SORT_LESSONS = new Set(["title", "level", "language", "createdAt",
"updatedAt"]);
```

```
export function parseListQuery(url: URL, kind: "words" | "lessons") {
  const q = (url.searchParams.get("q") || "").trim();
  const sortRaw = url.searchParams.get("sort") || "createdAt";
  const allowed = kind === "words" ? SORT_WORDS : SORT_LESSONS;
  const sort = allowed.has(sortRaw) ? sortRaw : "createdAt";
  const order: 1 | -1 = url.searchParams.get("order") === "asc" ? 1 : -1;
  const limitRaw = Number(url.searchParams.get("limit"));
  const limit = Number.isInteger(limitRaw) && limitRaw > 0 ?
Math.min(limitRaw, 100) : 0;
  const sortSpec: Record<string, 1 | -1> = { [sort]: order, _id: order };
  return { q, sort, order, limit, sortSpec };
}
```

Листинг А.8 – Справочники языков, частей речи и уровней

```
export type Option = { code: string; label: string };

export const STUDY_LANGUAGES: (Option & { speech: string })[] = [
  { code: "en", label: "английский", speech: "en-US" },
  { code: "de", label: "немецкий", speech: "de-DE" },
  { code: "fr", label: "французский", speech: "fr-FR" },
  { code: "es", label: "испанский", speech: "es-ES" },
];

export const NATIVE_LANGUAGES: (Option & { speech: string })[] = [
  { code: "ru", label: "русский", speech: "ru-RU" },
  { code: "be", label: "белорусский", speech: "be-BY" },
  { code: "uk", label: "украинский", speech: "uk-UA" },
];

export const ALL_LANGUAGES = [...STUDY_LANGUAGES, ...NATIVE_LANGUAGES];

export const PARTS_OF_SPEECH: Option[] = [
  { code: "noun", label: "существительное" },
  { code: "verb", label: "глагол" },
  { code: "adjective", label: "прилагательное" },
  { code: "adverb", label: "наречие" },
  { code: "pronoun", label: "местоимение" },
  { code: "numeral", label: "числительное" },
  { code: "preposition", label: "предлог" },
  { code: "conjunction", label: "союз" },
  { code: "interjection", label: "междометие" },
  { code: "phrase", label: "устойчивое выражение" },
];

export const LEVELS = ["A1", "A2", "B1", "B2", "C1", "C2"];

export const DEFAULT_TAGS = ["еда", "быт", "учёба", "работа", "путешествия",
"приветствия", "глаголы", "прилагательные"];

export function isStudyLanguage(code: string) {
  return STUDY_LANGUAGES.some((item) => item.code === code);
}

export function isNativeLanguage(code: string) {
  return NATIVE_LANGUAGES.some((item) => item.code === code);
}

export function languageLabel(code: string) {
  return ALL_LANGUAGES.find((item) => item.code === code)?.label || code;
```

```
}

export function speechLang(code: string) {
  return ALL_LANGUAGES.find((item) => item.code === code)?.speech || code;
}

export function plural(count: number, one: string, few: string, many: string)
{
  const mod10 = count % 10;
  const mod100 = count % 100;
  if (mod10 === 1 && mod100 !== 11) {
    return one;
  }
  if (mod10 >= 2 && mod10 <= 4 && (mod100 < 12 || mod100 > 14)) {
    return few;
  }
  return many;
}

export function partOfSpeechLabel(code: string) {
  return PARTS_OF_SPEECH.find((item) => item.code === code)?.label || code;
}

export function normalizePartOfSpeech(value: unknown) {
  const raw = String(value ?? "").trim().toLowerCase();
  if (!raw) {
    return "";
  }
  const found = PARTS_OF_SPEECH.find((item) => item.code === raw ||
item.label === raw);
  return found ? found.code : null;
}
```

Листинг А.9 – Модель пользователя

```
import mongoose, { Schema } from "mongoose";

const UserSchema = new Schema(
  {
    name: { type: String, required: true, trim: true, maxlength: 80 },
    email: { type: String, required: true, unique: true, lowercase: true,
trim: true },
    passwordHash: { type: String, required: true },
    nativeLanguage: { type: String, default: "ru" },
    targetLanguage: { type: String, default: "en" },
  },
  { timestamps: true }
);

export default mongoose.models.User || mongoose.model("User", UserSchema);
```

Листинг А.10 – Модель словарной записи

```
import mongoose, { Schema } from "mongoose";

const WordSchema = new Schema(
  {
    userId: { type: Schema.Types.ObjectId, ref: "User", required: true,
index: true },
```

```
    term: { type: String, required: true, trim: true, maxlength: 120 },
    translation: { type: String, required: true, trim: true, maxlength: 240
},
    transcription: { type: String, default: "", trim: true, maxlength: 80 },
    language: { type: String, default: "en", trim: true, maxlength: 8 },
    partOfSpeech: { type: String, default: "", trim: true, maxlength: 40 },
    example: { type: String, default: "", trim: true, maxlength: 400 },
    tags: { type: [String], default: [] },
    mastery: { type: Number, default: 0, min: 0, max: 5 },
    notes: { type: String, default: "", maxlength: 400 },
  },
  { timestamps: true }
);

WordSchema.index({ userId: 1, term: 1 });

export default mongoose.models.Word || mongoose.model("Word", WordSchema);
```

Листинг А.11 – Модель урока

```
import mongoose, { Schema } from "mongoose";

const LessonSchema = new Schema(
  {
    userId: { type: Schema.Types.ObjectId, ref: "User", required: true,
index: true },
    title: { type: String, required: true, trim: true, maxlength: 160 },
    content: { type: String, required: true, trim: true, maxlength: 8000 },
    language: { type: String, default: "en", trim: true, maxlength: 8 },
    level: { type: String, default: "A1", enum: ["A1", "A2", "B1", "B2",
"C1", "C2"] },
    words: [{ type: Schema.Types.ObjectId, ref: "Word" }],
  },
  { timestamps: true }
);

export default mongoose.models.Lesson || mongoose.model("Lesson",
LessonSchema);
```

Листинг А.12 – Чтение и преобразование словарной записи

```
import { isStudyLanguage, normalizePartOfSpeech } from "./catalog";
import { clampMastery, optionalString, parseTags, requireString } from
"./validate";

export function serializeWord(doc: Record<string, unknown>) {
  return {
    id: String(doc._id),
    term: doc.term,
    translation: doc.translation,
    transcription: doc.transcription,
    language: doc.language,
    partOfSpeech: doc.partOfSpeech,
    example: doc.example,
    tags: doc.tags,
    mastery: doc.mastery,
    notes: doc.notes,
    createdAt: doc.createdAt,
    updatedAt: doc.updatedAt,
```

```
  };
}

export function readWordFields(body: Record<string, unknown>, partial:
boolean) {
  const errors: string[] = [];
  const data: Record<string, unknown> = {};
  const has = (key: string) => body[key] !== undefined;

  if (!partial || has("term")) {
    const err = requireString(body.term, "Слово", 1, 120);
    if (err) errors.push(err);
    else data.term = String(body.term).trim();
  }
  if (!partial || has("translation")) {
    const err = requireString(body.translation, "Перевод", 1, 240);
    if (err) errors.push(err);
    else data.translation = String(body.translation).trim();
  }
  for (const [key, field, max] of [
    ["transcription", "Транскрипция", 80],
    ["example", "Пример", 400],
    ["notes", "Заметка", 400],
  ] as const) {
    if (!partial || has(key)) {
      const err = optionalString(body[key], field, max);
      if (err) errors.push(err);
      else data[key] = String(body[key] ?? "").trim();
    }
  }
  if (has("language")) {
    const language = String(body.language).trim();
    if (!isStudyLanguage(language)) errors.push("Язык: выберите значение из
списка");
    else data.language = language;
  }
  if (!partial || has("partOfSpeech")) {
    const pos = normalizePartOfSpeech(body.partOfSpeech);
    if (pos === null) errors.push("Часть речи: выберите значение из списка");
    else data.partOfSpeech = pos;
  }
  if (!partial || has("tags")) {
    data.tags = parseTags(body.tags);
  }
  if (!partial || has("mastery")) {
    data.mastery = clampMastery(body.mastery);
  }
  return { errors, data };
}
```

Листинг А.13 – Обработчик списка и создания слов

```
import { NextRequest, NextResponse } from "next/server";
import mongoose from "mongoose";
import { getCurrentUser, unauthorized } from "@/lib/auth";
import Word from "@/lib/models/Word";
import Lesson from "@/lib/models/Lesson";
import { escapeRegex, parseListQuery } from "@/lib/validate";
import { readWordFields, serializeWord } from "@/lib/words";
import { notify } from "@/lib/notify";
```

```
export const dynamic = "force-dynamic";

export async function GET(request: NextRequest) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const url = new URL(request.url);
  const { q, limit, sortSpec } = parseListQuery(url, "words");
  const filter: Record<string, unknown> = { userId: user._id };
  if (q) {
    const pattern = escapeRegex(q);
    filter.$or = [
      { term: { $regex: pattern, $options: "i" } },
      { translation: { $regex: pattern, $options: "i" } },
      { tags: { $regex: pattern, $options: "i" } },
    ];
  }
  const tag = (url.searchParams.get("tag") || "").trim().toLowerCase();
  if (tag) {
    filter.tags = tag;
  }
  const language = (url.searchParams.get("language") || "").trim();
  if (language) {
    filter.language = language;
  }
  const lessonId = url.searchParams.get("lesson") || "";
  if (lessonId) {
    if (!mongoose.Types.ObjectId.isValid(lessonId)) {
      return NextResponse.json({ error: "Урок не найден" }, { status: 404 });
    }
    const lesson = await Lesson.findOne({ _id: lessonId, userId: user._id
}).lean<{ words?: unknown[] }>();
    if (!lesson) {
      return NextResponse.json({ error: "Урок не найден" }, { status: 404 });
    }
    filter._id = { $in: lesson.words || [] };
  }
  let query = Word.find(filter).sort(sortSpec);
  if (limit) {
    query = query.limit(limit);
  }
  const items = await query.lean();
  return NextResponse.json({ items: items.map(serializeWord) });
}

export async function POST(request: NextRequest) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const body = await request.json().catch(() => ({}));
  const { errors, data } = readWordFields(body, false);
  if (errors.length) {
    return NextResponse.json({ error: errors.join("; ") }, { status: 400 });
  }

  const created = await Word.create({
    language: user.targetLanguage || "en",
    ...data,
    userId: user._id,
```

```
  });

  await notify(String(user._id), "Слово добавлено", `«${created.term}»
сохранено в словаре.`, "success");
  return NextResponse.json({ item: serializeWord(created.toObject()) }, {
status: 201 });
}
```

Листинг А.14 – Обработчик чтения, изменения и удаления слова

```
import { NextRequest, NextResponse } from "next/server";
import mongoose from "mongoose";
import { getCurrentUser, unauthorized } from "@/lib/auth";
import Word from "@/lib/models/Word";
import Lesson from "@/lib/models/Lesson";
import { readWordFields, serializeWord } from "@/lib/words";
import { notify } from "@/lib/notify";

export const dynamic = "force-dynamic";

function idFilter(userId: unknown, id: string) {
  if (!mongoose.Types.ObjectId.isValid(id)) {
    return null;
  }
  return { _id: id, userId };
}

export async function GET(_: NextRequest, { params }: { params: { id: string
} }) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const filter = idFilter(user._id, params.id);
  if (!filter) {
    return NextResponse.json({ error: "Запись не найдена" }, { status: 404
});
  }
  const item = await Word.findOne(filter).lean<Record<string, unknown>>();
  if (!item) {
    return NextResponse.json({ error: "Запись не найдена" }, { status: 404
});
  }
  return NextResponse.json({ item: serializeWord(item) });
}

export async function PATCH(request: NextRequest, { params }: { params: { id:
string } }) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const filter = idFilter(user._id, params.id);
  if (!filter) {
    return NextResponse.json({ error: "Запись не найдена" }, { status: 404
});
  }
  const body = await request.json().catch(() => ({}));
  const { errors, data } = readWordFields(body, true);
  if (errors.length) {
```

```
    return NextResponse.json({ error: errors.join("; ") }, { status: 400 });
  }
  const item = await Word.findOneAndUpdate(filter, data, { new: true
}).lean<Record<string, unknown>>();
  if (!item) {
    return NextResponse.json({ error: "Запись не найдена" }, { status: 404
});
  }
  const onlyMastery = Object.keys(data).length === 1 && "mastery" in data;
  if (!onlyMastery) {
    await notify(String(user._id), "Слово обновлено", `Изменения для
«${item.term}» сохранены.`, "info");
  }
  return NextResponse.json({ item: serializeWord(item) });
}

export async function DELETE(_: NextRequest, { params }: { params: { id:
string } }) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const filter = idFilter(user._id, params.id);
  if (!filter) {
    return NextResponse.json({ error: "Запись не найдена" }, { status: 404
});
  }
  const item = await Word.findOneAndDelete(filter).lean<Record<string,
unknown>>();
  if (!item) {
    return NextResponse.json({ error: "Запись не найдена" }, { status: 404
});
  }
  await Lesson.updateMany({ userId: user._id, words: item._id }, { $pull: {
words: item._id } });
  await notify(String(user._id), "Слово удалено", `«${item.term}» удалено из
словаря.`, "warning");
  return NextResponse.json({ ok: true });
}
```

Листинг А.15 – Чтение и преобразование урока

```
import { Types } from "mongoose";
import Word from "./models/Word";
import { LEVELS, isStudyLanguage } from "./catalog";
import { parseObjectIds, requireString } from "./validate";

type LessonDoc = Record<string, unknown> & { words?: unknown[] };
type WordRef = { _id: unknown; term: string; translation: string;
transcription?: string; language: string };

export async function serializeLessons(userId: unknown, docs: LessonDoc[]) {
  const ids = docs.flatMap((doc) => (doc.words || []).map(String));
  const refs = ids.length
    ? await Word.find({ userId, _id: { $in: ids } }, "term translation
transcription language").lean<WordRef[]>()
    : [];
  const byId = new Map(refs.map((ref) => [String(ref._id), ref]));
  return docs.map((doc) => ({
    id: String(doc._id),
```

```
    title: doc.title,
    content: doc.content,
    language: doc.language,
    level: doc.level,
    words: (doc.words || [])
      .map((id) => byId.get(String(id)))
      .filter((ref): ref is WordRef => Boolean(ref))
      .map((ref) => ({
        id: String(ref._id),
        term: ref.term,
        translation: ref.translation,
        transcription: ref.transcription || "",
        language: ref.language,
      })),
    createdAt: doc.createdAt,
    updatedAt: doc.updatedAt,
  }));
}

export async function readLessonFields(userId: unknown, body: Record<string,
unknown>, partial: boolean) {
  const errors: string[] = [];
  const data: Record<string, unknown> = {};
  const has = (key: string) => body[key] !== undefined;

  if (!partial || has("title")) {
    const err = requireString(body.title, "Название", 2, 160);
    if (err) errors.push(err);
    else data.title = String(body.title).trim();
  }
  if (!partial || has("content")) {
    const err = requireString(body.content, "Содержание", 10, 8000);
    if (err) errors.push(err);
    else data.content = String(body.content).trim();
  }
  if (!partial || has("level")) {
    const level = String(body.level || "A1").toUpperCase();
    if (!LEVELS.includes(level)) errors.push("Уровень: допустимы A1, A2, B1,
B2, C1, C2");
    else data.level = level;
  }
  if (has("language")) {
    const language = String(body.language).trim();
    if (!isStudyLanguage(language)) errors.push("Язык: выберите значение из
списка");
    else data.language = language;
  }
  if (!partial || has("words")) {
    const ids = parseObjectIds(body.words);
    const owned = ids.length ? await Word.find({ userId, _id: { $in: ids } },
"_id").lean<{ _id: Types.ObjectId }[]>() : [];
    const ownedSet = new Set(owned.map((item) => String(item._id)));
    data.words = ids.filter((id) => ownedSet.has(id));
  }
  return { errors, data };
}
```

Листинг А.16 – Обработчик перевода

```
import { NextRequest, NextResponse } from "next/server";
```

```
import { getCurrentUser, unauthorized } from "@/lib/auth";
import { ALL_LANGUAGES } from "@/lib/catalog";
import { requireString } from "@/lib/validate";

export const dynamic = "force-dynamic";

const CODES = new Set(ALL_LANGUAGES.map((item) => item.code));

function tidy(translation: string, source: string) {
  const text = translation.replace(/\s+/g, " ").trim();
  const shouting = (value: string) => /\p{L}/u.test(value) && value ===
value.toUpperCase();
  if (!shouting(text) || shouting(source)) {
    return text;
  }
  const lower = text.toLowerCase();
  return /^\p{Lu}/u.test(source) ? lower.charAt(0).toUpperCase() +
lower.slice(1) : lower;
}

export async function POST(request: NextRequest) {
  const user = await getCurrentUser();
  if (!user) {
    return unauthorized();
  }
  const body = await request.json().catch(() => ({}));
  const err = requireString(body.text, "Текст", 1, 500);
  if (err) {
    return NextResponse.json({ error: err }, { status: 400 });
  }
  const from = String(body.from || user.targetLanguage || "en").trim();
  const to = String(body.to || user.nativeLanguage || "ru").trim();
  if (!CODES.has(from) || !CODES.has(to)) {
    return NextResponse.json({ error: "Язык перевода не поддерживается" }, {
status: 400 });
  }
  if (from === to) {
    return NextResponse.json({ error: "Выберите разные языки перевода" }, {
status: 400 });
  }
  const source = String(body.text).trim();
  const endpoint = process.env.TRANSLATE_ENDPOINT ||
"https://api.mymemory.translated.net/get";
  const url =
`${endpoint}?q=${encodeURIComponent(source)}&langpair=${encodeURIComponent(`$
{from}|${to}`)}`;

  try {
    const res = await fetch(url, { cache: "no-store", signal:
AbortSignal.timeout(10000) });
    if (!res.ok) {
      return NextResponse.json({ error: "Сервис перевода временно недоступен"
}, { status: 502 });
    }
    const data = await res.json();
    const translation = tidy(String(data?.responseData?.translatedText ||
""), source);
    if (!translation || Number(data?.responseStatus) !== 200) {
      return NextResponse.json({ error: "Не удалось получить перевод" }, {
status: 502 });
    }
```

```
    return NextResponse.json({ translation, from, to, provider: "MyMemory"
});
  } catch {
    return NextResponse.json({ error: "Ошибка обращения к сервису перевода"
}, { status: 502 });
  }
}
```

Листинг А.17 – Модель уведомления

```
import mongoose, { Schema } from "mongoose";

const NoticeSchema = new Schema(
  {
    userId: { type: Schema.Types.ObjectId, ref: "User", required: true,
index: true },
    title: { type: String, required: true, maxlength: 120 },
    message: { type: String, required: true, maxlength: 400 },
    kind: { type: String, enum: ["info", "success", "warning"], default:
"info" },
    read: { type: Boolean, default: false },
  },
  { timestamps: true }
);

export default mongoose.models.Notice || mongoose.model("Notice",
NoticeSchema);
```

Листинг А.18 – Диспетчер подписчиков SSE

```
type Client = {
  userId: string;
  send: (payload: string) => void;
};

const clients = new Map<string, Set<Client>>();

export function subscribe(userId: string, send: (payload: string) => void) {
  const client: Client = { userId, send };
  const set = clients.get(userId) || new Set<Client>();
  set.add(client);
  clients.set(userId, set);
  return () => {
    set.delete(client);
    if (set.size === 0) {
      clients.delete(userId);
    }
  };
}

export function pushEvent(userId: string, data: unknown) {
  const payload = `data: ${JSON.stringify(data)}\n\n`;
  const set = clients.get(userId);
  if (!set) {
    return;
  }
  for (const client of set) {
    try {
      client.send(payload);
```

```
    } catch {
      set.delete(client);
    }
  }
}
```

Листинг А.19 – Создание и отправка уведомления

```
import Notice from "./models/Notice";
import { pushEvent } from "./sse";

export async function notify(
  userId: string,
  title: string,
  message: string,
  kind: "info" | "success" | "warning" = "info"
) {
  const notice = await Notice.create({ userId, title, message, kind });
  const payload = {
    id: String(notice._id),
    title: notice.title,
    message: notice.message,
    kind: notice.kind,
    read: false,
    createdAt: notice.createdAt,
  };
  pushEvent(userId, payload);
  return payload;
}
```

Листинг А.20 – Обработчик потока уведомлений

```
import { getCurrentUser } from "@/lib/auth";
import { subscribe } from "@/lib/sse";

export const dynamic = "force-dynamic";

export async function GET() {
  const user = await getCurrentUser();
  if (!user) {
    return new Response("Требуется авторизация", { status: 401 });
  }

  const encoder = new TextEncoder();
  let cleanup: () => void = () => undefined;

  const stream = new ReadableStream({
    start(controller) {
      const send = (payload: string) =>
controller.enqueue(encoder.encode(payload));
      cleanup = subscribe(String(user._id), send);
      send(`data: ${JSON.stringify({ type: "hello" })}\n\n`);
    },
    cancel() {
      cleanup();
    },
  });

  return new Response(stream, {
```

```
    headers: {
      "Content-Type": "text/event-stream; charset=utf-8",
      "Cache-Control": "no-cache, no-transform",
      Connection: "keep-alive",
    },
  });
}
```

Листинг А.21 – Макет личного кабинета

```
"use client";

import Link from "next/link";
import { usePathname, useRouter } from "next/navigation";
import { useEffect, useMemo, useRef, useState } from "react";
import { CabinetContext, CabinetUser } from "@/components/CabinetContext";

type Notice = { id: string; title: string; message: string; kind: string;
createdAt: string; read?: boolean };

const TOAST_TTL = 8000;

export default function CabinetLayout({ children }: { children:
React.ReactNode }) {
  const pathname = usePathname();
  const router = useRouter();
  const [user, setUser] = useState<CabinetUser | null>(null);
  const [toasts, setToasts] = useState<Notice[]>([]);
  const [notices, setNotices] = useState<Notice[]>([]);
  const timers = useRef(new Map<string, ReturnType<typeof setTimeout>>());

  useEffect(() => {
    fetch("/api/auth/me")
      .then((res) => {
        if (!res.ok) {
          throw new Error("auth");
        }
        return res.json();
      })
      .then((data) => setUser(data.user))
      .catch(() => router.replace("/login"));
  }, [router]);

  useEffect(() => {
    fetch("/api/notifications")
      .then((res) => res.json())
      .then((data) => setNotices(data.items || []))
      .catch(() => undefined);

    const source = new EventSource("/api/notifications/stream");
    source.onmessage = (event) => {
      const payload = JSON.parse(event.data);
      if (!payload.title) {
        return;
      }
      const notice = { ...payload, read: false };
      setToasts((prev) => [notice, ...prev].slice(0, 3));
      setNotices((prev) => [notice, ...prev].slice(0, 40));
      timers.current.set(notice.id, setTimeout(() => dismissToast(notice.id),
TOAST_TTL));
```

```
  };
  const pending = timers.current;
  return () => {
    source.close();
    pending.forEach((timer) => clearTimeout(timer));
    pending.clear();
  };
}, []);

const links = useMemo(
  () => [
    ["/cabinet", "Обзор"],
    ["/cabinet/words", "Слова"],
    ["/cabinet/lessons", "Уроки"],
    ["/cabinet/progress", "Прогресс"],
    ["/cabinet/translator", "Переводчик"],
    ["/cabinet/profile", "Профиль"],
  ],
  []
);

const unread = notices.filter((item) => !item.read).length;

async function logout() {
  await fetch("/api/auth/logout", { method: "POST" });
  router.replace("/");
}

function dismissToast(id: string) {
  const timer = timers.current.get(id);
  if (timer) {
    clearTimeout(timer);
    timers.current.delete(id);
  }
  setToasts((prev) => prev.filter((item) => item.id !== id));
}

async function markAllRead() {
  const res = await fetch("/api/notifications", { method: "PATCH" });
  if (!res.ok) {
    return;
  }
  setNotices((prev) => prev.map((item) => ({ ...item, read: true })));
  timers.current.forEach((timer) => clearTimeout(timer));
  timers.current.clear();
  setToasts([]);
}

return (
  <CabinetContext.Provider value={{ user, setUser }}>
    <div className="shell">
      <header className="topbar">
        <Link className="brand" href="/cabinet">
          <div className="mark">L</div>
          <div>
            <b>Lingua</b>
            <span>{user?.name || "личный кабинет"}</span>
          </div>
        </Link>
        <button className="btn secondary" onClick={logout} type="button">
          Выйти
        </button>
```

```
        </header>
        <nav className="cabinet-nav">
          {links.map(([href, label]) => (
            <Link className={pathname === href ? "active" : ""} href={href}
key={href}>
              {label}
            </Link>
          ))}
        </nav>
        {children}
        <aside className="card" style={{ marginTop: 18 }}>
          <div className="notice-head">
            <h3>
              Уведомления {unread ? <span className="badge">{unread}</span> :
null}
            </h3>
            <button className="btn ghost" type="button" onClick={markAllRead}
disabled={!unread}>
              Прочитать все
            </button>
          </div>
          <div className="notice-list">
            {notices.length === 0 ? <p>Пока нет событий.</p> : null}
            {notices.map((item) => (
              <p key={item.id} className={item.read ? "notice read" :
"notice"}>
                <b>{item.title}.</b> {item.message}
                <time>{new Date(item.createdAt).toLocaleString("ru-RU", {
dateStyle: "short", timeStyle: "short" })}</time>
              </p>
            ))}
          </div>
        </aside>
        <div className="toast-stack">
          {toasts.map((item) => (
            <div className={`toast ${item.kind}`} key={item.id}>
              <button className="toast-close" type="button" onClick={() =>
dismissToast(item.id)} aria-label="Скрыть">
                ×
              </button>
              <b>{item.title}</b>
              <div>{item.message}</div>
            </div>
          ))}
        </div>
      </div>
    </CabinetContext.Provider>
  );
}
```

Листинг А.22 – Компонент модального окна

```
"use client";

import { ReactNode, useEffect, useRef } from "react";

type Props = {
  open: boolean;
  title: string;
  onClose: () => void;
```

```
  children: ReactNode;
  wide?: boolean;
};

export default function Modal({ open, title, onClose, children, wide }:
Props) {
  const ref = useRef<HTMLDivElement>(null);
  const closeRef = useRef(onClose);
  closeRef.current = onClose;

  useEffect(() => {
    if (!open) {
      return;
    }
    const previous = document.activeElement as HTMLElement | null;
    const root = ref.current;
    const field =
      root?.querySelector<HTMLElement>("[data-autofocus]") ||
      root?.querySelector<HTMLElement>(".modal-body input:not([type=hidden]),
.modal-body select, .modal-body textarea");
    field?.focus();
    const onKey = (event: KeyboardEvent) => {
      if (event.key === "Escape") {
        closeRef.current();
      }
    };
    document.addEventListener("keydown", onKey);
    document.body.classList.add("modal-open");
    return () => {
      document.removeEventListener("keydown", onKey);
      document.body.classList.remove("modal-open");
      previous?.focus?.();
    };
  }, [open]);

  if (!open) {
    return null;
  }

  return (
    <div className="modal-backdrop" onMouseDown={(event) => event.target ===
event.currentTarget && onClose()}>
      <div className={wide ? "modal wide" : "modal"} ref={ref} role="dialog"
aria-modal="true" aria-label={title}>
        <div className="modal-head">
          <h3>{title}</h3>
          <button className="icon-btn" type="button" onClick={onClose} aria-
label="Закрыть">
            ×
          </button>
        </div>
        <div className="modal-body">{children}</div>
      </div>
    </div>
  );
}
```

Листинг А.23 – Компонент формы слова

```
"use client";
```

```
import { FormEvent, useEffect, useState } from "react";
import Modal from "./Modal";
import SpeakButton from "./SpeakButton";
import TagPicker from "./TagPicker";
import { useCabinet } from "./CabinetContext";
import { PARTS_OF_SPEECH, STUDY_LANGUAGES, languageLabel,
normalizePartOfSpeech } from "@/lib/catalog";

export type Word = {
  id: string;
  term: string;
  translation: string;
  transcription: string;
  language: string;
  partOfSpeech: string;
  example: string;
  tags: string[];
  mastery: number;
  notes: string;
};

type Draft = Omit<Word, "id">;

type Props = {
  open: boolean;
  word?: Word | null;
  preset?: Partial<Draft>;
  onClose: () => void;
  onSaved: (item: Word) => void;
};

function toDraft(word: Partial<Draft> | null | undefined, language: string):
Draft {
  return {
    term: word?.term || "",
    translation: word?.translation || "",
    transcription: word?.transcription || "",
    language: word?.language || language,
    partOfSpeech: normalizePartOfSpeech(word?.partOfSpeech) || "",
    example: word?.example || "",
    tags: word?.tags || [],
    mastery: word?.mastery ?? 0,
    notes: word?.notes || "",
  };
}

export default function WordForm({ open, word, preset, onClose, onSaved }:
Props) {
  const { user } = useCabinet();
  const native = user?.nativeLanguage || "ru";
  const [form, setForm] = useState<Draft>(() => toDraft(word || preset,
user?.targetLanguage || "en"));
  const [knownTags, setKnownTags] = useState<string[]>([]);
  const [error, setError] = useState("");
  const [busy, setBusy] = useState(false);
  const [translating, setTranslating] = useState<"" | "forward" |
"back">("");

  useEffect(() => {
    if (!open) {
      return;
```

```
    }
    setForm(toDraft(word || preset, user?.targetLanguage || "en"));
    setError("");
    fetch("/api/words/tags")
      .then((res) => res.json())
      .then((data) => setKnownTags(data.items || []))
      .catch(() => setKnownTags([]));
  }, [open, word]);

  function update<K extends keyof Draft>(key: K, value: Draft[K]) {
    setForm((prev) => ({ ...prev, [key]: value }));
  }

  async function translate(direction: "forward" | "back") {
    const text = direction === "forward" ? form.term : form.translation;
    if (!text.trim()) {
      return;
    }
    setTranslating(direction);
    setError("");
    const res = await fetch("/api/translate", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(
        direction === "forward" ? { text, from: form.language, to: native } :
{ text, from: native, to: form.language }
      ),
    }).catch(() => null);
    const data = res ? await res.json().catch(() => ({})) : {};
    setTranslating("");
    if (!res || !res.ok) {
      setError(data.error || "Перевод недоступен");
      return;
    }
    update(direction === "forward" ? "translation" : "term",
data.translation);
  }

  async function onSubmit(event: FormEvent) {
    event.preventDefault();
    setBusy(true);
    setError("");
    const res = await fetch(word ? `/api/words/${word.id}` : "/api/words", {
      method: word ? "PATCH" : "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form),
    }).catch(() => null);
    const data = res ? await res.json().catch(() => ({})) : {};
    setBusy(false);
    if (!res || !res.ok) {
      setError(data.error || "Ошибка сохранения");
      return;
    }
    onSaved(data.item);
  }

  const foreign = languageLabel(form.language);
  const nativeLabel = languageLabel(native);

  return (
    <Modal open={open} title={word ? `Редактирование слова «${word.term}»` :
"Новое слово"} onClose={onClose} wide>
```

```
      <form onSubmit={onSubmit}>
        <div className="form-row">
          <div>
            <label htmlFor="word-language">Язык слова</label>
            <select id="word-language" value={form.language} onChange={(e) =>
update("language", e.target.value)}>
              {STUDY_LANGUAGES.map((item) => (
                <option key={item.code} value={item.code}>
                  {item.label}
                </option>
              ))}
            </select>
          </div>
          <div>
            <label htmlFor="word-pos">Часть речи</label>
            <select id="word-pos" value={form.partOfSpeech} onChange={(e) =>
update("partOfSpeech", e.target.value)}>
              <option value="">не указана</option>
              {PARTS_OF_SPEECH.map((item) => (
                <option key={item.code} value={item.code}>
                  {item.label}
                </option>
              ))}
            </select>
          </div>
        </div>
        <div className="form-row">
          <div>
            <label htmlFor="word-term">Слово ({foreign})</label>
            <div className="input-with-action">
              <input id="word-term" value={form.term} onChange={(e) =>
update("term", e.target.value)} required maxLength={120} data-autofocus />
              <SpeakButton text={form.term} language={form.language} />
            </div>
          </div>
          <div>
            <label htmlFor="word-translation">Перевод ({nativeLabel})</label>
            <input id="word-translation" value={form.translation}
onChange={(e) => update("translation", e.target.value)} required
maxLength={240} />
          </div>
        </div>
        <div className="translate-actions">
          <button className="btn secondary small" type="button" onClick={()
=> translate("forward")} disabled={!form.term.trim() ||
Boolean(translating)}>
            {translating === "forward" ? "Перевод..." : `${foreign} →
${nativeLabel}`}
          </button>
          <button className="btn secondary small" type="button" onClick={()
=> translate("back")} disabled={!form.translation.trim() ||
Boolean(translating)}>
            {translating === "back" ? "Перевод..." : `${nativeLabel} →
${foreign}`}
          </button>
          <span className="hint">Автоперевод через сервис MyMemory в обе
стороны</span>
        </div>
        <div className="form-row">
          <div>
            <label htmlFor="word-transcription">Транскрипция</label>
```

```
            <input id="word-transcription" value={form.transcription}
onChange={(e) => update("transcription", e.target.value)} maxLength={80}
placeholder="например, ˈæpl" />
          </div>
          <div>
            <label>Уровень усвоения</label>
            <div className="segmented" role="radiogroup" aria-label="Уровень
усвоения">
              {[0, 1, 2, 3, 4, 5].map((level) => (
                <button
                  key={level}
                  type="button"
                  role="radio"
                  aria-checked={form.mastery === level}
                  className={form.mastery === level ? "active" : ""}
                  onClick={() => update("mastery", level)}
                >
                  {level}
                </button>
              ))}
            </div>
          </div>
        </div>
        <label htmlFor="word-example">Пример употребления</label>
        <input id="word-example" value={form.example} onChange={(e) =>
update("example", e.target.value)} maxLength={400} />
        <label>Теги</label>
        <TagPicker value={form.tags} known={knownTags} onChange={(tags) =>
update("tags", tags)} />
        <label htmlFor="word-notes">Заметка</label>
        <textarea id="word-notes" className="short" value={form.notes}
onChange={(e) => update("notes", e.target.value)} maxLength={400}
placeholder="ассоциация, правило, особенность произношения" />
        {error ? <p className="error">{error}</p> : null}
        <div className="modal-actions">
          <button className="btn ghost" type="button" onClick={onClose}>
            Отмена
          </button>
          <button className="btn" disabled={busy} type="submit">
            {busy ? "Сохранение..." : word ? "Сохранить изменения" :
"Добавить слово"}
          </button>
        </div>
      </form>
    </Modal>
  );
}
```

Листинг А.24 – Компонент выбора тегов

```
"use client";

import { KeyboardEvent, useMemo, useState } from "react";
import { DEFAULT_TAGS } from "@/lib/catalog";

type Props = {
  value: string[];
  known: string[];
  onChange: (tags: string[]) => void;
};
```

```
export default function TagPicker({ value, known, onChange }: Props) {
  const [draft, setDraft] = useState("");
  const suggestions = useMemo(() => {
    const all = Array.from(new Set([...known, ...DEFAULT_TAGS]));
    return all.filter((tag) => !value.includes(tag)).sort((a, b) =>
a.localeCompare(b, "ru"));
  }, [known, value]);

  function add(raw: string) {
    const tag = raw.trim().toLowerCase().slice(0, 30);
    if (tag && !value.includes(tag) && value.length < 12) {
      onChange([...value, tag]);
    }
    setDraft("");
  }

  function onKey(event: KeyboardEvent<HTMLInputElement>) {
    if (event.key === "Enter" || event.key === ",") {
      event.preventDefault();
      add(draft);
    }
    if (event.key === "Backspace" && !draft && value.length) {
      onChange(value.slice(0, -1));
    }
  }

  return (
    <div className="tag-picker">
      <div className="chips">
        {value.map((tag) => (
          <span className="chip" key={tag}>
            {tag}
            <button type="button" onClick={() => onChange(value.filter((item)
=> item !== tag))} aria-label={`Убрать тег ${tag}`}>
              ×
            </button>
          </span>
        ))}
        <select
          className="chip-select"
          value=""
          onChange={(event) => add(event.target.value)}
          aria-label="Выбрать тег из списка"
          disabled={!suggestions.length || value.length >= 12}
        >
          <option value="">+ выбрать тег</option>
          {suggestions.map((tag) => (
            <option key={tag} value={tag}>
              {tag}
            </option>
          ))}
        </select>
      </div>
      <input
        value={draft}
        onChange={(event) => setDraft(event.target.value)}
        onKeyDown={onKey}
        onBlur={() => draft && add(draft)}
        placeholder="или введите новый тег и нажмите Enter"
        maxLength={30}
        disabled={value.length >= 12}
```

```
      />
    </div>
  );
}
```

Листинг А.25 – Компонент озвучивания

```
"use client";

import { useEffect, useState } from "react";
import { speechLang } from "@/lib/catalog";

type Props = { text: string; language: string; label?: string };

export default function SpeakButton({ text, language, label }: Props) {
  const [supported, setSupported] = useState(false);
  const [speaking, setSpeaking] = useState(false);

  useEffect(() => {
    setSupported(typeof window !== "undefined" && "speechSynthesis" in
window);
  }, []);

  function speak() {
    const phrase = text.trim();
    if (!supported || !phrase) {
      return;
    }
    const synth = window.speechSynthesis;
    const lang = speechLang(language);
    const utterance = new SpeechSynthesisUtterance(phrase);
    utterance.lang = lang;
    utterance.rate = 0.9;
    const prefix = lang.slice(0, 2).toLowerCase();
    const voice = synth.getVoices().find((item) =>
item.lang.toLowerCase().startsWith(prefix));
    if (voice) {
      utterance.voice = voice;
    }
    utterance.onend = () => setSpeaking(false);
    utterance.onerror = () => setSpeaking(false);
    synth.cancel();
    setSpeaking(true);
    synth.speak(utterance);
  }

  const title = supported ? label || "Озвучить" : "Браузер не поддерживает
озвучивание";

  return (
    <button
      className={speaking ? "icon-btn speak active" : "icon-btn speak"}
      type="button"
      onClick={speak}
      disabled={!supported || !text.trim()}
      title={title}
      aria-label={title}
    >
      <svg viewBox="0 0 24 24" width="18" height="18" aria-hidden="true">
        <path d="M4 9v6h4l5 4V5L8 9H4z" fill="currentColor" />
```

```
        <path d="M16 8.5a5 5 0 0 1 0 7M18.5 6a8.5 8.5 0 0 1 0 12" fill="none"
stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" />
      </svg>
    </button>
  );
}
```

Листинг А.26 – Страница прогресса и тренировки

```
"use client";

import { FormEvent, useEffect, useMemo, useRef, useState } from "react";
import SpeakButton from "@/components/SpeakButton";
import { languageLabel, plural } from "@/lib/catalog";

type Stats = {
  wordsTotal: number;
  lessonsTotal: number;
  learned: number;
  byMastery: { level: number; count: number }[];
  byLanguage: Record<string, number>;
};

type Word = { id: string; term: string; translation: string; transcription:
string; example: string; language: string; mastery: number };
type LessonOption = { id: string; title: string; words: unknown[] };
type Direction = "term" | "translation";
type Result = { ok: boolean; expected: string } | null;

function variants(value: string) {
  return value
    .split(/[,;]/)
    .map((item) => item.trim().toLowerCase())
    .filter(Boolean);
}

export default function ProgressPage() {
  const [stats, setStats] = useState<Stats | null>(null);
  const [lessons, setLessons] = useState<LessonOption[]>([]);
  const [source, setSource] = useState("");
  const [direction, setDirection] = useState<Direction>("term");
  const [words, setWords] = useState<Word[]>([]);
  const [current, setCurrent] = useState<Word | null>(null);
  const [answer, setAnswer] = useState("");
  const [result, setResult] = useState<Result>(null);
  const [hint, setHint] = useState(false);
  const [score, setScore] = useState({ right: 0, total: 0 });
  const answerRef = useRef<HTMLInputElement>(null);

  async function loadStats() {
    const res = await fetch("/api/progress");
    setStats(await res.json());
  }

  async function loadWords(lesson: string) {
    const params = new URLSearchParams({ sort: "mastery", order: "asc" });
    if (lesson) {
      params.set("lesson", lesson);
    }
    const res = await fetch(`/api/words?${params}`);
```

```
    const data = await res.json();
    return (data.items || []) as Word[];
  }

  useEffect(() => {
    loadStats().catch(() => undefined);
    fetch("/api/lessons?sort=title&order=asc")
      .then((res) => res.json())
      .then((data) => setLessons(data.items || []))
      .catch(() => undefined);
    const lesson = new URLSearchParams(window.location.search).get("lesson")
|| "";
    setSource(lesson);
  }, []);

  useEffect(() => {
    let cancelled = false;
    loadWords(source)
      .then((items) => {
        if (cancelled) {
          return;
        }
        setWords(items);
        setScore({ right: 0, total: 0 });
        pick(items, null);
      })
      .catch(() => !cancelled && setWords([]));
    return () => {
      cancelled = true;
    };
  }, [source]);

  const maxMastery = useMemo(() => Math.max(1, ...(stats?.byMastery.map((x)
=> x.count) || [1])), [stats]);

  function pick(pool: Word[], previous: Word | null) {
    if (!pool.length) {
      setCurrent(null);
      return;
    }
    const weak = pool.filter((w) => w.mastery < 5);
    let candidates = weak.length ? weak : pool;
    if (previous && candidates.length > 1) {
      candidates = candidates.filter((w) => w.id !== previous.id);
    }
    setCurrent(candidates[Math.floor(Math.random() * candidates.length)]);
    setAnswer("");
    setResult(null);
    setHint(false);
    setTimeout(() => answerRef.current?.focus(), 0);
  }

  async function check(event: FormEvent) {
    event.preventDefault();
    if (!current || result) {
      return;
    }
    const expected = direction === "term" ? current.translation :
current.term;
    const ok = variants(expected).includes(answer.trim().toLowerCase());
    setResult({ ok, expected });
```

```
    setScore((prev) => ({ right: prev.right + (ok ? 1 : 0), total: prev.total
+ 1 }));
    const mastery = Math.max(0, Math.min(5, current.mastery + (ok ? 1 : -
1)));
    if (mastery === current.mastery) {
      return;
    }
    const res = await fetch(`/api/words/${current.id}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ mastery }),
    });
    if (res.ok) {
      const updated = { ...current, mastery };
      setWords((prev) => prev.map((w) => (w.id === updated.id ? updated :
w)));
      setCurrent(updated);
      await loadStats();
    }
  }

  const prompt = current ? (direction === "term" ? current.term :
current.translation) : "";
  const hintText = !current
    ? ""
    : direction === "term"
      ? current.example || "Для этого слова нет примера употребления."
      : `Первая буква «${current.term.charAt(0)}», символов:
${current.term.length}.`;

  return (
    <div>
      <div className="stats">
        <div className="stat">
          <b>{stats?.wordsTotal ?? 0}</b>
          {plural(stats?.wordsTotal ?? 0, "слово", "слова", "слов")} всего
        </div>
        <div className="stat">
          <b>{stats?.learned ?? 0}</b>
          освоено (4–5)
        </div>
        <div className="stat">
          <b>{stats?.lessonsTotal ?? 0}</b>
          {plural(stats?.lessonsTotal ?? 0, "урок", "урока", "уроков")}
        </div>
        <div className="stat">
          <b className="small-stat">
            {Object.entries(stats?.byLanguage || {}).length
              ? Object.entries(stats?.byLanguage || {}).map(([k, v]) => (
                  <span key={k}>
                    {languageLabel(k)} — {v}
                  </span>
                ))
              : "—"}
          </b>
          по языкам
        </div>
      </div>

      <div className="grid two">
        <article className="card">
          <h3>Усвоение</h3>
```

```
          {(stats?.byMastery || []).map((row) => (
            <p key={row.level} className="hist-row">
              <span>уровень {row.level}</span>
              <span className="bar">
                <span style={{ width: `${(row.count / maxMastery) * 100}%` }}
/>
              </span>
              <span>{row.count}</span>
            </p>
          ))}
        </article>
        <article className="card">
          <div className="review-head">
            <h3>Повторение</h3>
            <span className="muted">
              верно {score.right} из {score.total}
            </span>
          </div>
          <div className="form-stack">
            <div>
              <label htmlFor="review-source">Слова</label>
              <select id="review-source" value={source} onChange={(e) =>
setSource(e.target.value)}>
                <option value="">весь словарь</option>
                {lessons.map((lesson) => (
                  <option key={lesson.id} value={lesson.id}
disabled={!lesson.words.length}>
                    урок «{lesson.title}» ({lesson.words.length})
                  </option>
                ))}
              </select>
            </div>
            <div>
              <label>Направление</label>
              <div className="segmented wide" role="radiogroup" aria-
label="Направление тренировки">
                <button type="button" role="radio" aria-checked={direction
=== "term"} className={direction === "term" ? "active" : ""} onClick={() => {
setDirection("term"); pick(words, null); }}>
                  слово → перевод
                </button>
                <button type="button" role="radio" aria-checked={direction
=== "translation"} className={direction === "translation" ? "active" : ""}
onClick={() => { setDirection("translation"); pick(words, null); }}>
                  перевод → слово
                </button>
              </div>
            </div>
          </div>
          {current ? (
            <form onSubmit={check} className="review-card">
              <p className="muted small">{direction === "term" ? `Переведите
с языка «${languageLabel(current.language)}»` : `Вспомните слово на языке
«${languageLabel(current.language)}»`}</p>
              <div className="term big">
                <b>{prompt}</b>
                {direction === "term" ? <SpeakButton text={current.term}
language={current.language} /> : null}
              </div>
              {direction === "term" && current.transcription ? <p
className="muted">[{current.transcription}]</p> : null}
              {hint ? <p className="hint-box">{hintText}</p> : null}
```

```
              <input ref={answerRef} value={answer} onChange={(e) =>
setAnswer(e.target.value)} placeholder={direction === "term" ? "перевод" :
"слово"} disabled={Boolean(result)} />
              {result ? (
                <p className={result.ok ? "result ok" : "result bad"}>
                  {result.ok ? "Верно!" : `Неверно. Правильный ответ:
${result.expected}`}
                  {!result.ok || direction === "translation" ? <SpeakButton
text={current.term} language={current.language} /> : null}
                </p>
              ) : null}
              <p className="nav">
                {result ? (
                  <button className="btn" onClick={() => pick(words,
current)} type="button" autoFocus>
                    Следующее
                  </button>
                ) : (
                  <>
                    <button className="btn" type="submit"
disabled={!answer.trim()}>
                      Проверить
                    </button>
                    <button className="btn secondary" onClick={() =>
setHint(true)} type="button" disabled={hint}>
                      Подсказка
                    </button>
                    <button className="btn ghost" onClick={() => pick(words,
current)} type="button">
                      Пропустить
                    </button>
                  </>
                )}
              </p>
            </form>
          ) : (
            <p>{source ? "В уроке нет слов для повторения." : "Добавьте
слова, чтобы начать повторение."}</p>
          )}
        </article>
      </div>
    </div>
  );
}
```